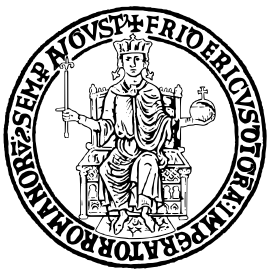

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

IMPACT OF UNSUPERVISED CODE FINE-TUNING ON LLMS' REASONING

Relatrice
Prof.ssa Anna CORAZZA

Candidata
Teresa Pia DE SANTIS
N86/4617

Anno Accademico 2024–2025

Riassunto

Questa tesi analizza se il fine-tuning non supervisionato su codice sorgente possa migliorare le capacità di ragionamento dei large language models anche in compiti non legati alla programmazione. L'obiettivo è verificare l'esistenza del *Code-Induced Transfer Effect* (CITE), ossia il beneficio aggiuntivo che deriva dal fine-tuning su codice rispetto a un fine-tuning equivalente su testo in linguaggio naturale.

Per studiare questo effetto in modo controllato, vengono confrontati modelli della famiglia Pythia adattati con diverse tecniche di parameter-efficient fine-tuning (IA3, VeRA, Prefix-Tuning e LoRA), usando dataset di codice tratti da StarCoderData e, come termine di confronto, testo in linguaggio naturale proveniente da FineWeb-Edu. La valutazione è stata condotta su benchmark di ragionamento matematico, logico e di commonsense, tra cui GSM8K, ANLI, BBH, ARC-Easy, BoolQ e PIQA.

I risultati mostrano che il CITE esiste, ma non in modo uniforme. Il fine-tuning su codice può produrre benefici in alcuni compiti di ragionamento al di fuori della programmazione, ma l'effetto dipende fortemente dal metodo di adattamento e dal task considerato, mentre il ruolo della scala del modello risulta meno chiaro nel setup sperimentale adottato. In particolare, LoRA è il metodo che più spesso produce effetti positivi, mentre i diversi linguaggi di programmazione non mostrano differenze sistematiche marcate.

In conclusione, il codice può favorire il trasferimento di capacità di ragionamento, ma non rappresenta una soluzione generale. Il suo impatto va quindi interpretato come condizionato dal contesto sperimentale, dal tipo di aggiornamento applicato al modello e dal benchmark utilizzato.

Contents

1. Introduction

1.1 Motivation

Large language models (LLMs) now demonstrate impressive reasoning capabilities, from solving algebra to handling complex logic. These abilities are not usually taught explicitly; rather, they emerge during training.

Recent evidence suggests that reasoning performance depends not only on model scale, but also on the type of data used during training. In particular, studies have found that exposure to source code can improve reasoning performance beyond programming tasks, with models trained on both natural language and code often outperforming purely text-trained counterparts on mathematics and logic benchmarks [1, 2, 3]. This raises a deeper question: what exactly does code contribute?

Unlike natural language, code is strictly structured, compositional, and governed by formal rules. Exposure to code may therefore encourage models to internalize patterns that support step-by-step reasoning and symbolic manipulation.

Current research investigates this phenomenon during massive-scale pretraining, or through supervised fine-tuning with labeled reasoning tasks. However, both settings make it difficult to isolate what code itself contributes: large-scale pretraining mixes code with many other signals and is expensive to reproduce on limited hardware; supervised fine-tuning adds explicit task guidance that can improve reasoning regardless of any structural benefit from code. This motivates studying code exposure in a more controlled adaptation setting, with the goal of identifying whether code alone produces transferable gains in non-programming reasoning.

In this work, we refer to this phenomenon as the *Code-Induced Transfer Effect* (CITE): the incremental improvement in non-programming reasoning tasks that is specifically attributable to exposure to source code. CITE does not denote general gains from adaptation; rather, it captures the contribution of code structure compared to natural-language training under the same fine-tuning setup.

The central objective of this thesis is to determine whether CITE exists and to identify the conditions under which it emerges. The following research questions break this objective into testable components.

1.2 Research Questions

This study investigates whether unsupervised code fine-tuning can produce a measurable CITE in domains unrelated to programming. To structure this inquiry, we examine the following research questions:

RQ1. Does unsupervised fine-tuning on source code produce a measurable CITE in math, logic, and commonsense reasoning tasks?

RQ2. Does the programming language used during fine-tuning modulate the magnitude or direction of the CITE?

RQ3. How does the choice of fine-tuning method influence the strength and consistency of the CITE?

RQ4. How does model scale affect the emergence and magnitude of the CITE?

RQ5. Which reasoning tasks are most affected by the CITE?

Together, these questions aim to disentangle three core factors:

(1) the structural properties of training data,

(2) the fine-tuning method used, and

(3) the capacity of the underlying model.

By systematically varying these dimensions, this study seeks to determine whether reasoning improvements stem from the intrinsic structure of code, or whether they are mainly driven by supervised signals and the many confounding factors present in large-scale pretraining.

1.3 Thesis Overview

Chapter 2 provides the background and related work needed to motivate the study. It introduces large language models in terms of their observable capabilities, then reviews pretraining and adaptation, with a focus on how different data types shape downstream behavior. The chapter closes by positioning code-induced transfer as a question that remains difficult to isolate in large-scale or supervised settings.

Chapter 3 outlines the design choices of the study. It motivates the use of unsupervised code fine-tuning, describes the selected Pythia base models, and introduces the parameter-efficient fine-tuning (PEFT) methods used in the experiments. It then details the code and natural-language datasets and the reasoning benchmarks used to measure CITE across math, logic, and commonsense domains.

Chapter 4 describes the experimental setup and pipeline. It reports the hardware and software environment, explains how experiments are configured and logged, and details the training procedure and evaluation protocol used to ensure fair, comparable runs across the full experimental grid.

Chapter 5 presents the empirical results. It defines the core metrics, including Δ^*, and reports findings for each research question. This chapter focuses on measurement and comparison only, without interpretation.

Chapter 6 discusses the results. It interprets the patterns observed in Chapter 5, highlights what appears consistent versus conditional, and summarizes limitations and implications.

Chapter 7 concludes the thesis. It answers the research questions directly, summarizes the contributions, and outlines limitations and directions for future work.

The appendix provides raw evaluation results and configuration-level breakdowns for transparency.

2. Background and Related Work

2.1 What Large Language Models Are and Do

Large language models (LLMs) are neural language models trained on large text corpora to predict the next token in a sequence. At a high level, an LLM can be described as a parameterized function that maps a sequence of input tokens to a probability distribution over possible next tokens. Prior to training, the model consists solely of randomly initialized parameters and exhibits no meaningful behavior. Through training on vast text corpora, it learns statistical regularities of language, including lexical usage, syntactic structure, and semantic relationships.

Because this training objective is general rather than task-specific, LLMs can often be applied to a wide range of downstream tasks with little or no task-specific modification. These tasks include dialogue, summarization, translation, question answering, and code generation. As model scale and training data have increased, LLMs have also shown improving performance on tasks that involve mathematical reasoning, logical inference, and commonsense judgment.

These skills are not usually directly taught through direct supervision during pretraining. Instead, they are often treated as emergent effects of scale, data composition, and model design, although the precise mechanisms underlying their emergence remain an active topic of research.

2.2 Pretraining and Adaptation

LLM development typically involves two phases: initial broad training (pretraining) and later refinement (adaptation or fine-tuning). Pretraining gives the model broad linguistic and world knowledge by optimizing a general objective over large and diverse corpora. Adaptation then adjusts a pretrained model so that it performs better in a particular domain, style, or task setting, usually with far less data and computation than would be needed to train a model from scratch.

Adaptation methods differ in the degree of guidance they provide. Supervised approaches use labeled examples, such as question–answer pairs, to guide the model toward desired outputs. They are effective for well-defined tasks but can lead models to memorize superficial patterns rather than build deeper understanding.

Unsupervised adaptation, on the other hand, continues the original pretraining objective (next-token prediction) on targeted corpora without explicit labels. This method shifts the model's internal representations toward new styles, structures, or domains while preserving its broad capabilities, making it especially useful for domain extension or refinement.

Adaptation can also vary in efficiency: parameter-efficient fine-tuning (PEFT) methods update only a small subset of parameters while freezing the rest. This enables targeted continued training with minimal computational cost and low risk of catastrophic forgetting, making them ideal for isolating data-driven effects, such as those from structured code exposure, without heavy resource demands.

2.3 The Role of Data in Shaping Capabilities

The choice of training data plays a central role in shaping model behavior. Datasets differ not only in what they explicitly contain, but also in the kinds of inductive bias they introduce during training. Narrative text emphasizes discourse and style, factual corpora support retrieval and recall, and source code provides a unique framework of logical structure and symbolic operations.

Prior work has shown that exposure to code during large-scale pretraining improves performance in non-programming domains such as natural language reasoning and mathematics, even when these abilities are not the explicit target [1].

This phenomenon suggests that code acts as a form of cognitive scaffolding [4]. Because code requires discrete, executable steps and verifiable logic, it encourages the model to develop high-level abstractions that are functionally similar to the requirements of formal logic and commonsense reasoning.

Recent work has begun to explore whether these advantages can be elicited during lighter adaptation stages. One approach involves code-augmented fine-tuning, where source code is paired with natural language comments to bridge the gap between structural logic and multilingual reasoning [2]. Another method is instruction fine-

tuning (IFT), which uses coding tasks to activate reasoning in symbolic and arithmetic contexts [3].

However, both of these approaches introduce supervised task-specific signals, which can make it harder to isolate the contribution of code's inherent structure. Furthermore, intensive supervised fine-tuning can trigger negative transfer, where improvements in a narrow reasoning task come at the expense of the model's general-purpose capabilities and flexibility [5].

Together, these results motivate the CITE question studied in this thesis: whether code can induce reasoning transfer under lightweight, unsupervised fine-tuning, rather than only during large-scale pretraining or supervised training.

3. Design Choices

3.1 Unsupervised Fine-Tuning

We choose unsupervised over supervised adaptation to cleanly isolate CITE from supervised task signals. Supervised methods introduce task-specific cues through labeled examples, which could obscure whether improvements stem from the data itself or from the provided labels.

Unsupervised adaptation keeps the intervention clean: it isolates code's structural properties, allowing the model to internalize logical flow, state management, and computational patterns.

3.2 Model Selection

We use models from the Pythia suite [6] as base LLMs for our unsupervised fine-tuning experiments. Pythia was chosen for several reasons that directly support the goals of this study.

First, the suite offers exceptional transparency and reproducibility. All Pythia models are trained on the publicly available Pile corpus [7] using identical training procedures, data ordering, and openly released code and checkpoints. This setup minimizes confounds from pretraining, letting us attribute changes to code adaptation alone, unlike closed models with unverifiable data. In particular, it allows us to precisely account for the models' prior exposure to code, including the GitHub subset contained in The Pile (which constitutes approximately 7–8% of the effective training weight).

Second, Pythia employs a decoder-only autoregressive architecture trained purely on next-token prediction, which is the same objective we retain during unsupervised fine-tuning on source code. This architectural consistency is key, as it allows the model to adapt to the structured nature of code while staying faithful to its original pretraining paradigm. As a result, Pythia's general-purpose language modeling capabilities remain largely preserved, making any observed improvements in non-programming reasoning

tasks more directly attributable to code's structural properties rather than to shifts in training dynamics or loss objectives.

Third, the Pythia suite spans multiple model scales, ranging from 70M to 12B parameters. This range enables controlled analysis of whether potential transfer effects from code fine-tuning depend on model capacity. Since scaling is known to influence emergent behaviors in large language models [8], examining multiple sizes allows us to assess whether reasoning improvements vary as a function of parameter count.

Our experiments therefore cover a spectrum of capacities: Pythia-160M and Pythia-1B serve as the core models, with full combinations of PEFT methods, programming languages, and the natural language baseline run on both to ensure robust comparisons at small and medium scales. Pythia-2.8B is included as an exploratory check at higher capacities, though with more limited runs due to increased computational demands. While these sizes remain modest relative to contemporary frontier models (which frequently exceed 70B parameters), they enable efficient investigation of scaling trends within lightweight, resource-constrained fine-tuning setups (RQ4).

3.3 PEFT Methods

In this study, we use parameter-efficient fine-tuning (PEFT) as the primary way to expose models to code. Prior work has shown that reasoning-related capabilities can emerge or improve during large-scale pretraining, instruction fine-tuning, and other forms of full-model adaptation, as discussed in Chapter 2. However, it remains underexplored whether similar transfer effects can arise under lightweight adaptation. PEFT allows us to test this by exposing models to structured code while modifying only a small fraction of parameters and largely preserving the original pretrained weights.

To capture a diverse range of adaptation mechanisms, we select four representative PEFT methods: IA3, VeRA, Prefix-Tuning and LoRA. These methods differ in their intervention strategy and resource footprint:

- **IA3** [9] learns small scaling vectors that modulate attention and feed-forward activations ($\sim$0.01–0.05% added parameters).
- **VeRA** [10] learns small trainable vectors added directly to selected weight matrices ($\sim$0.01–0.1% added parameters).

- **Prefix-Tuning** [11] injects trainable prefix vectors into each layer's attention mechanism (∼0.1–0.5% of original parameters).
- **LoRA** [12] (Low-Rank Adaptation) adds trainable low-rank matrices to selected weight matrices (∼0.1–1% trainable parameters).

These methods were selected according to three main criteria:

- **Conceptual diversity**: each relies on a different adaptation strategy.
- **Strong presence in the literature**: all are widely used and well-established in the PEFT community.
- **Compatibility** with decoder-only models and next-token prediction unsupervised objectives.

Together, these methods form a spectrum of intervention strength. Moving from IA3 to LoRA, we increase the number of trainable parameters, the degree of structural modification, and the associated computational cost. This spectrum allows us to examine whether reasoning transfer from code fine-tuning depends on the magnitude and nature of parameter updates (RQ3).

3.4 Fine-Tuning Datasets

We perform unsupervised continued pretraining on two main types of data.

3.4.1 Source Code Data

We use subsets of the StarCoderData corpus [13, 14], derived from The Stack [15].

StarCoderData is a large, deduplicated and filtered collection of permissively licensed source code (≈783 GB) covering 86 programming languages.

Our experiments use four code fine-tuning datasets drawn from StarCoderData: three single-language subsets (Python, Java, and C++), and one mixed dataset (Multi) that we construct by combining Java, Python, C, C++, and SQL.

This setup lets us compare transfer from single-language code exposure against transfer from a more structurally diverse multi-language mixture. The selected languages vary in typing, abstraction level, and programming style:

- **Python** uses relatively concise, high-level syntax.
- **Java** emphasizes explicit types and object-oriented structure.
- **C and C++** expose lower-level control over program state.
- **SQL** expresses logic in a declarative relational form.

This diversity supports RQ2, which asks whether the programming language used during fine-tuning modulates the CITE.

3.4.2 Natural Language Data

To disentangle the effects of code from generic benefits of additional adaptation, we include a high-quality natural language baseline. This baseline allows us to measure CITE as the differential gain of code fine-tuning relative to matched text fine-tuning.

We select FineWeb-Edu [16, 17], the educational subset of the FineWeb corpus. It comprises 1.3 trillion tokens of curated, high-educational-value English web text. The content is coherent, explanatory, and logically clear, and parallels the cleaned, high-quality nature of StarCoderData.

3.5 Evaluation Benchmarks

To assess CITE in non-programming domains, we evaluate models on three reasoning areas: mathematics, formal logic, and commonsense reasoning. We select benchmarks that require complex reasoning and contextual inference rather than factual recall, as our objective is to find out whether the logical patterns learned from code carry over to other kinds of reasoning.

For mathematical reasoning, we use:

- **GSM8K-CoT (Grade School Math 8K)** [18]: 1,319 grade-school math word problems requiring multi-step arithmetic reasoning. The model has to break the problem down, do intermediate calculations, and arrive at an exact numerical answer.

For logical reasoning, we use:

- **BBH (Big-Bench Hard)** [19]: we focus on the Logical Deduction and Boolean Expressions subsets (250 examples each). Logical Deduction asks the model to figure out properties or relationships between objects based on clues. Boolean Expressions tests symbolic logic with operators (and, or, not) and truth conditions.
- **ANLI (Adversarial Natural Language Inference)** [20]: 9,600 natural language inference examples where the model decides whether a hypothesis is entailed by, contradicts, or is neutral to a given premise. The dataset is built to be tricky and avoid easy shortcuts, so it really probes logical understanding.

Finally, for commonsense reasoning we use:

- **ARC-Easy (AI2 Reasoning Challenge – Easy)** [21]: 9,501 elementary science questions that combine basic facts with light reasoning.
- **BoolQ** [22]: 6,540 yes/no questions based on short passages, blending reading comprehension with everyday knowledge.
- **PIQA (Physical Interaction: Question Answering)** [23]: 3,676 physical commonsense reasoning questions that require selecting the more plausible solution to a real-world scenario.

4. Experimental Setup

4.1 Introduction to Experimental Setup

To facilitate numerous comparable experiments with reliability and reproducibility, a modular experimental pipeline was developed. The complete codebase and configuration files are publicly available on GitHub[1].

The setup is built around a centralized configuration system. This allows for quick adjustments to the study's three core variables: the base model, the PEFT method, and the dataset. Instead of rewriting code for every new test, any parameter can be changed through a simple command-line interface. This ensures that every experiment, from initialization to final evaluation, follows an identical, standardized workflow. A full run is triggered via a single execution:

```
python3 run_experiment.py
    model=pythia-1b
    peft=lora-pythia
    dataset=starcoderdata-python
```

The rest of this chapter details the experimental setup following the logical order of the pipeline's execution:

- The hardware and software used for training (§ 4.2).
- How experiments are set up (§ 4.3).
- Preparation of the model, PEFT adapter, and dataset (§ 4.4).
- Trainer configuration and fine-tuning procedure (§ 4.5).
- Model finalization, checkpointing, and logging (§ 4.6)
- The evaluation framework used to measure reasoning performance (§ 4.7)

[1] https://github.com/mmurido/code-to-reasoning-study

4.2 Hardware and Software Environment

4.2.1 Hardware Used

Experiments were run in a mixed environment to handle the workload efficiently:

- **University Server**: A Linux partition at Università degli Studi di Napoli Federico II, equipped with dual NVIDIA GeForce GTX 1080 GPUs (8.5 GB VRAM each).
- **Cloud Platforms**: Google Colab, Kaggle, and Lightning AI, utilizing NVIDIA Tesla T4 GPUs (16 GB VRAM).

All experiments used identical training code, random seeds, and hyperparameters. Hardware differences (Pascal for GTX 1080, Turing for T4) therefore do not explain any observed differences in results.

4.2.2 Software Used

The implementation is built on top of the PyTorch framework and the Hugging Face ecosystem, which provide standardized tools for model loading, tokenization, training, and evaluation. The specific library versions reported in Table 4.1 were selected for stability under CUDA 11.8 and kept fixed across all experiments to ensure reproducibility.

Library	Version	Role in Project	Reference
`torch`	`2.1.2`	Tensor computations and GPU acceleration	[24]
`transformers`	`4.48.0`	Model loading, tokenization, architecture	[25]
`peft`	`0.14.0`	Parameter-efficient adapters (LoRA, IA3, etc.)	[26]
`trl`	`0.13.0`	Supervised fine-tuning and training loops	[27]
`lm-eval`	`0.4.9`	Standardized reasoning benchmark evaluation	[28]
`hydra-core`	`1.3.2`	Hierarchical configuration management	[29]

Table 4.1: Core software dependencies and their roles.

4.3 Pipeline Initialization

Before model loading or training begins, the pipeline initializes each experiment by parsing the configuration and creating a dedicated, self-contained output directory. This design ensures that every run is reproducible, isolated, and fully traceable, which is essential for a controlled comparison across models, datasets, and PEFT methods.

4.3.1 Configuration Management

To manage the intersection of models, datasets, and PEFT methods, the pipeline utilizes Hydra, a hierarchical configuration framework. This enables a modular architecture where configuration is organized into two complementary categories:

- **Experimental Variables**: Individual configuration files that specify the model architecture, PEFT method parameters, and dataset composition. These components vary across experiments and are marked as required (via `???` in the root config), meaning they must be provided as command-line overrides at runtime.
- **Base Configs**: Shared defaults for training and evaluation that ensure consistency across all comparisons. These are applied automatically unless explicitly overridden.

This design is particularly important for this study, as it enables systematic and controlled comparisons across model scales, datasets, and PEFT methods without modifying the underlying training pipeline.

4.3.2 Experiment Directory Setup

Upon execution, the pipeline generates a unique `run_id` following the convention `<model>__<peft>__<dataset>__<timestamp>`. This identifier acts as the root of a structured output directory that stores checkpoints, logs, and evaluation results, effectively serving as a complete logbook for each experiment.

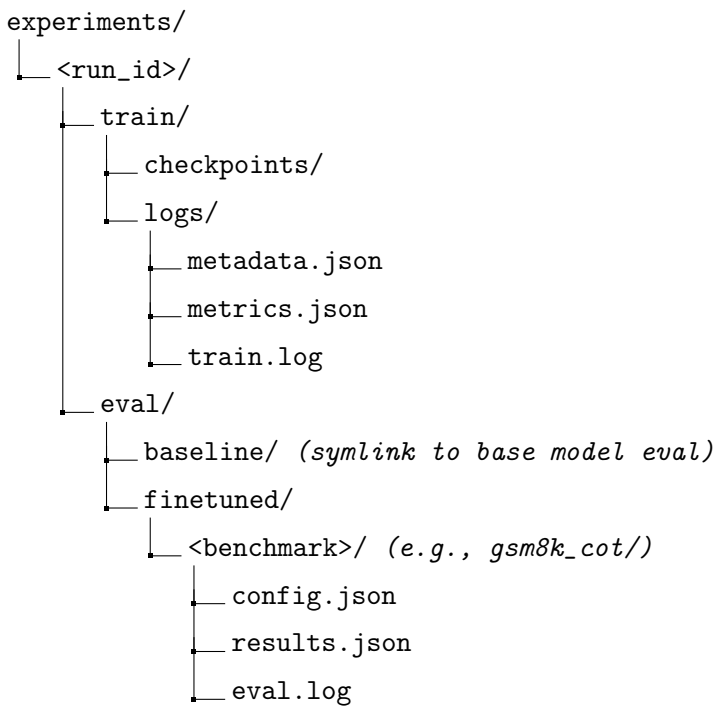

Figure 4.1: Standardized directory structure for an experimental run.

4.4 Model and Data Preparation

Once the runtime environment is initialized, the pipeline loads the base model, tokenizer, and dataset. The following subsections describe these preparation steps.

4.4.1 Loading the Base Model and Tokenizer

All pretrained Pythia causal language models are loaded from the Hugging Face Hub (see Section 3.2 for rationale on model family and scale selection). Given that the goal of this work is to study CITE under unsupervised code fine-tuning, no supervised alignment or instruction tuning is applied before training.

The tokenizer and model are loaded jointly using the `transformers` library. The end-of-sequence (EOS) token is reused as the padding token. This prevents the need to add new tokens or resize the model's vocabulary, keeping everything consistent and compatible across different model sizes and checkpoints.

Models are loaded in half-precision (FP16) with automatic device mapping, which distributes the model across available GPUs. This reduces memory usage significantly

and speeds up training. Key-value (KV) caching is disabled during training to save additional VRAM and ensure correct gradient computation.

Fixed loading parameters are summarized in Table 4.2.

Setting	**Configuration**
Model source	Hugging Face Hub (Pythia family)
Prior alignment	None
Tokenizer padding	EOS token reused
Precision	FP16 (half-precision)
Device mapping	Automatic (multi-GPU support)
KV caching	Disabled during training

Table 4.2: Fixed model and tokenizer loading parameters.

4.4.2 Building the PEFT Adapter

Adapters are constructed with the Hugging Face PEFT library, with the specific method selected via configuration (see Section 3.3 for motivation and comparison of adaptation strategies). All adapters are instantiated for the causal language modeling task (`task_type="CAUSAL_LM"`), matching the unsupervised next-token prediction objective. Target modules are set to `query_key_value` (attention) and `dense` (MLP) layers across all methods, as these are where the main representational transformations happen.

Table 4.3 summarizes the exact configurations used.

Method	Key Parameters	Value / Setting
LoRA	Rank (`r`)	16
	Scaling factor (`alpha`)	32
	Dropout	0.05
	Bias	none
	Target modules	`query_key_value`, `dense`
IA3	Target modules	`query_key_value`, `dense`
	Feed-forward modules	`dense`
	`fan_in_fan_out`	false
Prefix-Tuning	Number of virtual tokens	48
	Prefix projection	false
	Encoder hidden size	null
VeRA	Rank (`r`)	256
	Initial scaling (`d_initial`)	0.05
	Dropout	0.0
	Target modules	`query_key_value`, `dense`

Table 4.3: PEFT adapter configurations used in all experiments.

These values were chosen based on preliminary experiments and common practices in the PEFT literature, balancing expressivity with training efficiency.

4.4.3 Loading and Preparing the Data

We load training data from the Hugging Face Hub using the Datasets library in streaming mode (see Section 3.4 for rationale on dataset selection). This approach allows for processing very large corpora without the need to download or store the full dataset locally.

To guarantee a controlled and comparable training budget across all experiments, each run is assigned a nominal budget of 50 million tokens, regardless of the source dataset or the number of subsets selected. This number was selected to provide enough data for the model to adapt to code structures while remaining small enough to run dozens of comparative experiments on our hardware. By keeping this budget fixed, we ensure

that any differences in CITE are attributable to data composition, fine-tuning method, or model scale, rather than to the amount of data seen.

For the Multi condition, we construct the dataset ourselves by combining five StarCoderData subsets (Java, Python, C, C++, and SQL) into a single mixed training stream. Multi is therefore an experimental mixture defined for this study, rather than a native subset of StarCoderData. The nominal 50 million token budget is divided evenly across these five components, which are then interleaved with uniform probability.

The loading and subsampling process follows these fixed steps:

- The total token budget of 50 million is divided evenly across the chosen subsets (e.g., different programming languages).
- Each subset is streamed and tokenized on-the-fly until its per-subset token target would be exceeded, at which point streaming for that subset stops. As a result, realized token counts are approximately, though not always exactly, equal to the nominal target.
- When multiple subsets are used, they are interleaved with uniform probability and the process continues until each subset has been streamed up to its token cap.
- The preparation follows a one-pass strategy, where data is streamed linearly without repetition. Once a document is processed, it is not revisited, simulating a continuous pretraining environment.
- Every example is reduced to a single "text" field and used directly for causal language modeling, with no task-specific formatting, labels, or instruction templates added.

Fixed parameters and procedural choices are summarized in Table 4.4 for quick reference.

Parameter / Step	**Setting / Value**
Loading mode	Streaming
Sampling Strategy	One-pass (no replacement/repetition)
Total token budget	50,000,000 tokens
Per-subset allocation	Even split
Token counting	On-the-fly with model tokenizer
Multi-subset interleaving	Uniform probability
Stopping strategy	Stop before per-subset token target would be exceeded
Final example format	Single "text" field (no labels or formatting)

Table 4.4: Fixed dataset loading and preparation parameters.

4.5 Training Procedure

We use the Hugging Face TRL library (Transformer Reinforcement Learning), and specifically its SFTTrainer for unsupervised continued pretraining. Although the name implies *Supervised Fine-Tuning*, the trainer is used here strictly as a wrapper for causal language modeling to leverage its efficient PEFT integration and sequence packing. No instruction templates, labels, or response masking were applied; the training objective remains purely unsupervised next-token prediction.

All hyperparameters are fixed across the full experimental grid (see Table 4.5). This controlled setup ensures that any differences in downstream reasoning performance can be attributed to model scale, PEFT method, or data structure, and not to variations in optimization or training dynamics.

As noted in Section 4.4, the base models were loaded in half-precision (FP16) to optimize memory usage and allow for larger batch sizes. However, the training process itself was conducted in full precision (FP32). This choice was made to ensure numerical stability and consistent behavior across the different PEFT architectures. In preliminary tests, certain methods exhibited sensitivity to lower-precision gradients; moving to FP32 for the training loop eliminated these inconsistencies and ensured a fair comparison across all methods in the grid.

The most distinctive choices are:

- Each run lasts exactly 2,000 steps to ensure a uniform number of gradient updates.
- Effective batch size is 16 (per-device batch size 1 + 16 accumulation steps).
- Packing is used to maximize throughput within the 256-token context window.
- Learning rate follows cosine decay after 100 warmup steps.
- Gradient checkpointing is active to keep memory usage manageable during back-propagation.

All remaining parameters follow the standard `SFTTrainer` defaults unless explicitly overridden in the table.

Table 4.5: Fixed training hyperparameters.

Group	**Parameter**	**Value**
Reproducibility	Seed	42
Batch Configuration	Per-device train batch size	1
	Gradient accumulation steps	16
	Effective batch size	16
Optimization	Max training steps	2000
	Learning rate	5.0×10^{-5}
	Warmup steps	100
	LR scheduler	cosine
	Weight decay	0.01
Memory & Performance	FP16	false
	Max gradient norm	1.0
	Gradient checkpointing	true
	Dataloader workers	0
	Max sequence length	256
	Packing	true
Checkpointing & Logging	Logging steps	128
	Save steps	500
	Save total limit	4

4.6 Logging, Metadata, and Checkpointing

To ensure full reproducibility and traceability, the training pipeline logs all steps and automatically saves outputs in a consistent, structured format.

At the start of training, all console outputs are redirected to a persistent `train.log` file. This log captures the full execution trace, including dataset loading diagnostics, resolved configuration values, hardware information, model statistics, PEFT configuration, and the complete set of trainer arguments. As a result, each experiment maintains a human-readable record of the exact runtime conditions under which training was executed.

In parallel, a structured `metadata.json` file is generated as a snapshot of the experiment state at initialization. This file contains the fully resolved Hydra configuration, trainer arguments, PEFT configuration, and model statistics (including total and trainable parameter counts).

During training, the trainer's internal logging history (e.g., loss, gradient norm, learning rate, and step index) is continuously accumulated and later exported. Upon completion, a `metrics.json` file is created containing runtime duration, start and end timestamps, the full training log history, and the final training loss.

Checkpointing follows a step-based strategy consistent across all experiments. Intermediate checkpoints are saved every 500 steps, with a retention limit to control disk usage. After training terminates, the final adapter weights and tokenizer are explicitly saved to a dedicated checkpoint directory.

4.7 Evaluation Setup

Model performance is evaluated using the EleutherAI Language Model Evaluation Harness through its `lm_eval` interface. The harness runs standardized benchmarks in few-shot mode (see Section 3.5 for task selection and rationale).

Both baseline (pre-fine-tuning) and PEFT-adapted models are assessed under identical inference conditions. For adapted models, the base checkpoint is loaded from the Hugging Face Hub and the trained adapter is attached via the corresponding PEFT configuration. Baseline evaluation uses the same base model without any adapter.

Models are loaded in half precision with automatic device placement for efficient multi-GPU inference.

Evaluation proceeds task-by-task. Each task runs independently in its own output directory, producing a dedicated log file (`eval.log`), configuration snapshot (`config.json`), and report of results (`results.json`). If results already exist for a task, evaluation is skipped to avoid overwriting and allow safe resumption.

All tasks use the same generation parameters (Table 4.6): deterministic greedy decoding with 3 few-shot examples, 256 max new tokens, temperature 0.0, and early stopping enabled.

Parameter	**Value**
Batch size	16
Few-shot examples	3
Max new tokens	256
Temperature	0.0
Top-p	1.0
Sampling	Disabled
Beam search	1 beam (greedy)
Early stopping	Enabled

Table 4.6: Shared evaluation configuration across all experiments.

This per-task isolation keeps evaluation artifacts modular and traceable. Baseline and fine-tuned results remain in separate subdirectories but follow identical logic, enabling direct, fair comparison.

5. Results

5.1 Overview

This section summarizes the experimental matrix and defines the core metrics used throughout the chapter.

For completeness and transparency, the full set of raw benchmark accuracies and standard errors for all configurations is provided in Table A.1.

The remainder of this chapter presents the results organized by research question. Each section reports empirical findings only; interpretation is reserved for Chapter 6.

5.1.1 Experimental Matrix

The experimental design systematically varies four dimensions:

Dimension	**Levels**	**Count**
Model scale (primary)	Pythia-160M, Pythia-1B	2
PEFT method	IA3, VeRA, Prefix-Tuning, LoRA	4
Fine-tuning dataset	StarCoderData subsets (Java, Python, C++, and a Multi-language mixture of Java, Python, C, C++, and SQL); FineWeb-Edu (natural language baseline)	5
Evaluation task	Math: GSM8K; Logic: ANLI, BBH Boolean, BBH Logical; Commonsense: ARC-Easy, BoolQ, PIQA	7

Table 5.1: Experimental dimensions and resulting configuration counts

All main results are based on Pythia-160M and Pythia-1B only; experiments on Pythia-2.8B were run for a limited subset of configurations as an exploratory scale probe, and are reported in Section 5.5.

For each of the two primary models we ran:

- Code fine-tuning: 4 PEFT methods $\times$ 4 code datasets = 16 fine-tuned models
- Text fine-tuning: 4 PEFT methods $\times$ 1 dataset = 4 fine-tuned models

This results in 20 fine-tuned models per base model.

Each fine-tuned model was evaluated on all 7 reasoning tasks, resulting in 140 task evaluations per model (280 across both).

Every configuration was evaluated relative to the pretrained baseline on the same 7 tasks (14 baseline evaluations total), resulting in 294 evaluations overall.

5.1.2 Core Metrics

All results are reported in terms of benchmark accuracy (*Acc*). For any task, we first compute the fine-tuning gain relative to the pretrained baseline:

$$\Delta = Acc_{\text{FT}} - Acc_{\text{PT}} \tag{5.1}$$

where Acc_{PT} is the accuracy of the pretrained model and Acc_{FT} is the accuracy after fine-tuning.

To isolate the effect of code exposure beyond generic adaptation, we compare the fine-tuning gain from code against a matched natural language fine-tuning baseline. This differential gain defines the Code-Induced Transfer Effect (CITE):

$$\Delta^* = \Delta_{\text{Code}} - \Delta_{\text{Text}} \tag{5.2}$$

A positive Δ^* indicates that code fine-tuning yields larger gains than text fine-tuning on the same task, while a negative value indicates the opposite. We classify Δ^* as neutral when $|\Delta^*| \leq \varepsilon$ with $\varepsilon = 0.001$. Throughout this chapter, Δ^* is the primary unit of analysis.

To summarize trends across experimental dimensions, we report Δ^* aggregated in two ways:

- **Mean** Δ^*: simple average over a specified set of configurations (e.g., across programming languages or PEFT methods).
- **Macro** Δ^*: task-averaged CITE for a single configuration:

$$\Delta^*_{\text{macro}} = \frac{1}{T}\sum_{t=1}^{T}\Delta^*_t \tag{5.3}$$

  where T is the number of evaluation tasks. This gives equal weight to each benchmark, preventing any single task from dominating the summary.

5.2 RQ1: Overall CITE

RQ1 asks whether code fine-tuning produces a CITE on reasoning tasks relative to a natural-language fine-tuning baseline.

Operationally, we treat this as a global question: we pool results across the full experimental matrix and ask whether Δ^* shows an overall tendency to be positive, negative or near zero.

In this section we do not condition on any single moderator (task, PEFT method, dataset, or model size); those factors are analyzed in RQ2–RQ5.

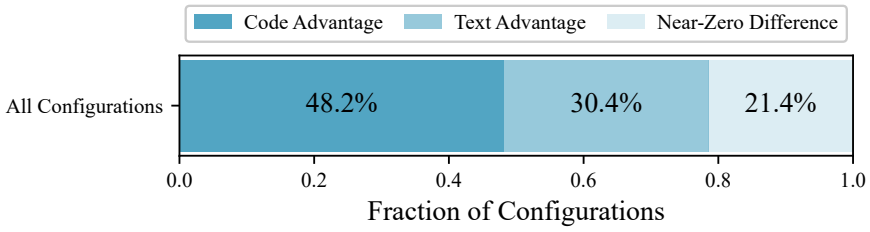

Figure 5.1: Stacked bar chart of Δ^* direction across configurations (near-zero difference: $|\Delta^*| \leq 0.001$).

Figure 5.1 summarizes the direction of Δ^* across all 56 model–PEFT–task configurations. Code advantage occurs in 27/56 cases (48.2%), text advantage in 17/56 (30.4%), and 12/56 (21.4%) show near-zero difference.

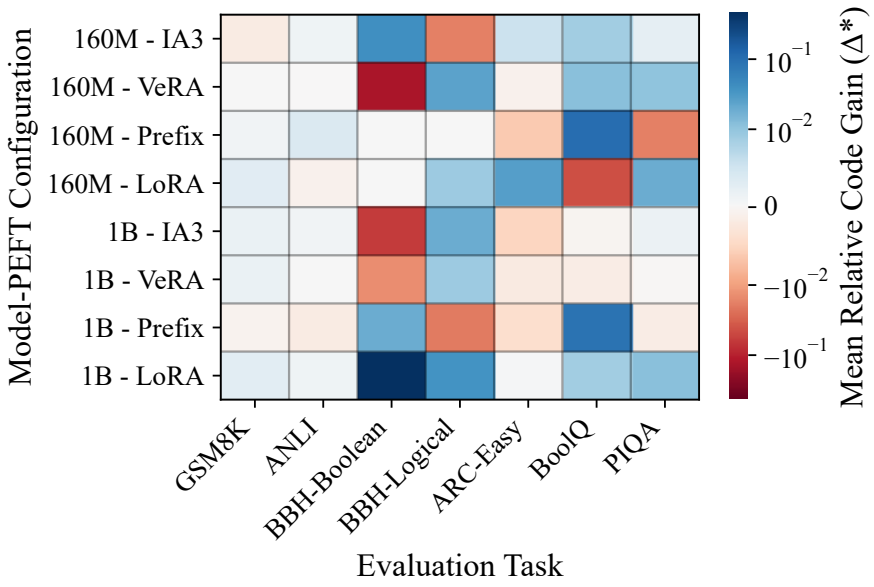

Figure 5.2: Heatmap of mean Δ^* across evaluation tasks.

Figure 5.2 localizes mean Δ^* within the matrix. Rows correspond to model–PEFT configurations and columns to evaluation tasks. Each cell shows Δ^* with Δ_{Code} averaged over the four code datasets and compared against the matched natural-language baseline for the same model and PEFT method. Blue indicates a code advantage ($\Delta^* > 0$), red indicates a text advantage ($\Delta^* < 0$), and white indicates near-zero difference ($|\Delta^*| \leq 0.001$).

Row-wise patterns (model–PEFT):

- **Pythia-160M:**
  - IA3 is mostly positive with two negative cells (GSM8K, BBH Logical).
  - VeRA is mixed and contains the most negative cell (BBH Boolean, $\Delta^* = -0.133$).
  - Prefix-Tuning is mostly near zero, with negatives on ARC-Easy and PIQA, and one large positive on BoolQ ($\Delta^* = 0.094$).
  - LoRA is mixed, including a moderate negative on BoolQ ($\Delta^* = -0.046$).

- **Pythia-1B:**
  - IA3 and VeRA remain small in magnitude with mixed signs.
  - Prefix-Tuning includes negatives on ANLI and BBH Logical, and a large positive on BoolQ ($\Delta^* = 0.083$).
  - LoRA is predominantly positive and contains the largest positive cell (BBH Boolean, $\Delta^* = 0.432$).

Column-wise patterns (tasks):

- **GSM8K, ANLI:** values are small and clustered near zero.
- **BBH Boolean:** widest spread, containing both the largest positive and most negative cells (0.432 and -0.133).
- **BBH Logical:** mostly positive (5/8), with two negatives and one zero.
- **ARC-Easy:** more negative than positive cells (5/8 negative), with few positives.
- **BoolQ:** mostly positive (5/8), including two large positives under Prefix-Tuning (0.094, 0.083) and one moderate negative under 160M+LoRA (-0.046).
- **PIQA:** mostly positive (5/8), with small magnitudes.

Overall, Δ^* varies across both tasks and model–PEFT configurations, with the largest deviations concentrated in a small subset of cells (primarily BBH Boolean and selected BoolQ entries).

5.3 RQ2: Programming Language Comparison

RQ2 asks whether the programming language used during code fine-tuning modulates the CITE.

Operationally, we hold the evaluation setting fixed (model, PEFT method, and task) and vary only the code dataset (Java, Python, C++, Multi), comparing the resulting Δ^* values against the same natural-language baseline.

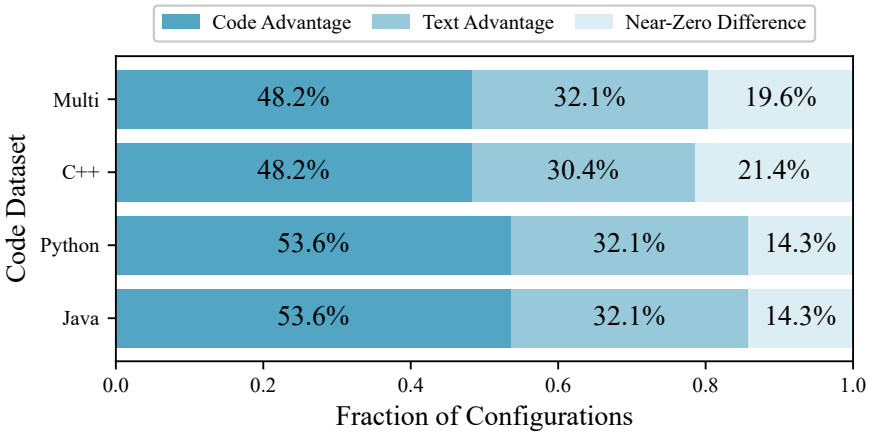

Figure 5.3: Stacked bar chart of Δ^* direction across programming language datasets (near-zero difference: $|\Delta^*| \leq 0.001$).

Figure 5.3 compares the direction of Δ^* across code datasets under a fixed near-zero cutoff ($|\Delta^*| \leq 0.001$). It provides a dataset-level view of how often fine-tuning on each language subset yields code advantage, text advantage, or near-zero differences.

Java and Python exhibit identical direction proportions (53.6% code advantage, 32.1% text advantage, 14.3% near zero). C++ and Multi show a lower share of code advantage (48.2%) and higher near-zero fractions (21.4% and 19.6%, respectively).

Across all 224 code configurations (56 model–PEFT–task pairs $\times$ 4 datasets), $\Delta^* > 0$ occurs in 114 cases (50.9%), $\Delta^* < 0$ in 71 cases (31.7%), and 39 cases (17.4%) are near zero.

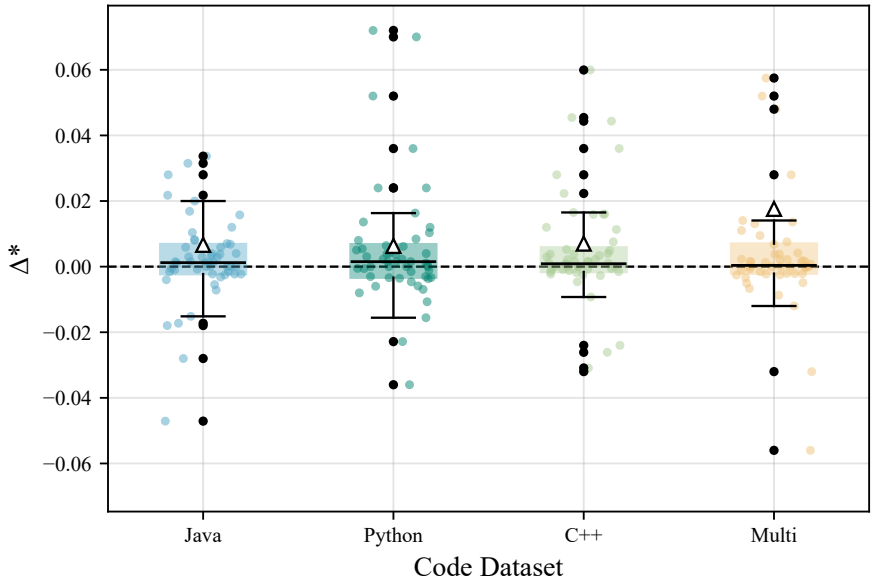

Figure 5.4: Boxplot of Δ^* by programming language dataset.

Figure 5.4 complements the direction summary by showing the full distribution of Δ^* values for each code dataset. While the stacked plot reports how often Δ^* falls above or below the cutoff, the boxplots show typical magnitudes and the extent of variation within each language subset.

- Medians are slightly positive for all four datasets and remain very small (0.0004–0.0015).
- Means are also positive. Java, Python, and C++ are similar (0.006–0.007), while the Multi-language subset is higher (0.0176).
- Dispersion is large relative to these means: standard deviations range from 0.048 (Python) to 0.089 (Multi), indicating both positive and negative values within each dataset.

Overall, the four language subsets exhibit broadly similar distributions, with the Multi-language dataset showing the highest mean and largest spread.

5.4 RQ3: PEFT Methods Comparison

RQ3 asks how the choice of PEFT method affects the CITE.

Operationally, we summarize CITE at the configuration level using Δ^*_{macro} (task-averaged) and compare its direction and magnitude across PEFT methods, pooling over models and code datasets.

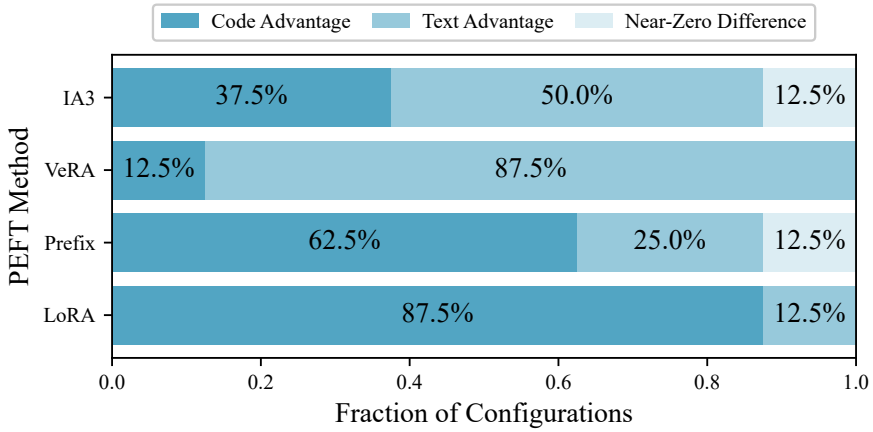

Figure 5.5: Stacked bar chart of Δ^*_{macro} direction across PEFT methods (near-zero difference: $|\Delta^*_{\text{macro}}| \leq 0.001$).

Figure 5.5 summarizes how often each PEFT method yields code advantage, text advantage, or near-zero differences under the fixed cutoff.

LoRA shows the highest frequency of code advantage (87.5%), while VeRA shows the highest frequency of text advantage (87.5%). Prefix-Tuning yields code advantage in 62.5% of cases. IA3 is more balanced, with 37.5% code advantage, 50.0% text advantage, and 12.5% near-zero difference.

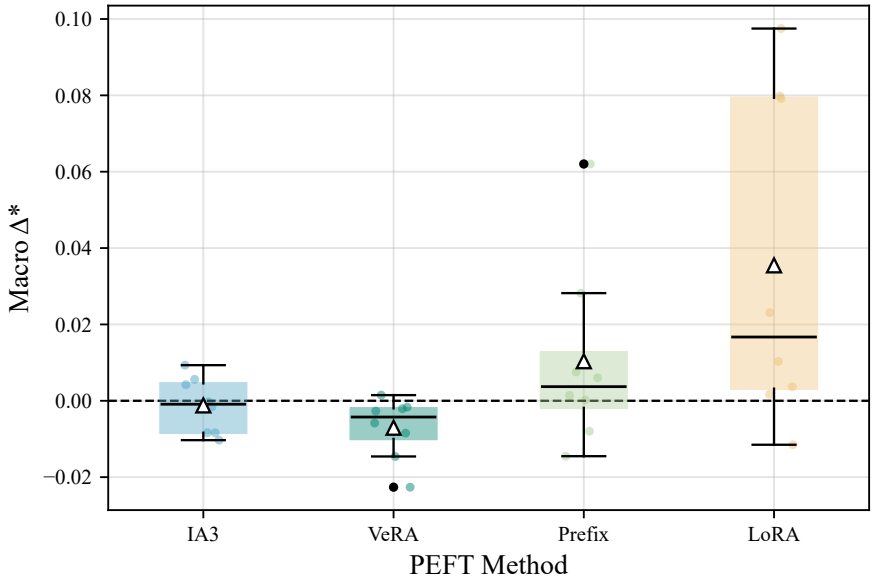

Figure 5.6: Boxplot of Δ^*_{macro} by PEFT method.

Figure 5.6 complements the direction summary by showing the magnitude and spread of Δ^*_{macro} values within each PEFT method.

- IA3 is centered near zero (mean -0.0012, median -0.0009) with a narrow range (-0.0103 to 0.0093).
- VeRA is shifted below zero (mean -0.0071, median -0.0043), with the box fully below zero and one negative outlier (-0.0226).
- Prefix-Tuning is modestly positive on average (mean 0.0104, median 0.0037) and includes one positive outlier (0.0620).
- LoRA has the highest central values (mean 0.0355, median 0.0167) and the largest spread overall (range -0.0115 to 0.0975), with the box above zero but whiskers extending across zero.

5.5 RQ4: Model Scale Comparison

RQ4 asks how model size affects the CITE.

We compare the same fine-tuning configurations across Pythia-160M, Pythia-1B, and Pythia-2.8B. Because fine-tuning the 2.8B model is substantially more expensive, we did not replicate the full matrix from the smaller models. Consequently, our analysis is restricted to "matched triplets", which are configurations where the same PEFT method and code dataset were successfully run across all three model sizes.

In total, 13 such triplets were evaluated. The missing 2.8B code configurations are Prefix-Tuning + Java, VeRA + Python, and IA3 + Python.

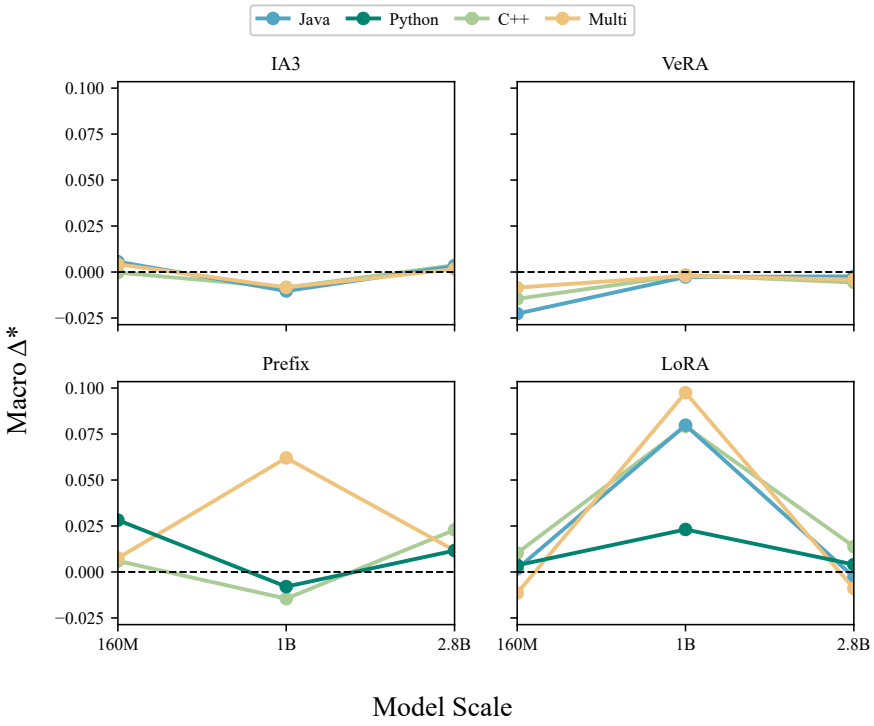

Figure 5.7: Matched slope plot of Δ^*_{macro} across model scales, faceted by PEFT method and colored by code dataset.

Figure 5.7 tracks Δ^*_{macro} across scales for each matched (PEFT, dataset) triplet. Each line corresponds to one configuration and shows how the task-averaged CITE changes from 160M to 1B to 2.8B.

The IA3 panel shows nearly overlapping trajectories: all three configurations move from slightly positive at 160M to negative at 1B and return close to their starting value at 2.8B, forming a consistent V-shaped pattern.

VeRA shows the opposite monotonic trend: values improve from 160M to 1B and then remain similar at 2.8B, with all three trajectories staying below zero across scales.

Prefix-Tuning exhibits mixed behavior across datasets. The C++ and Python configurations form V-shaped trajectories (positive at 160M, negative at 1B, positive again at 2.8B), whereas the Multi-language configuration shows an inverted-V shape (improving strongly at 1B and partially reverting at 2.8B while remaining positive).

LoRA displays the largest scale-dependent swings. All LoRA trajectories rise sharply from 160M to 1B and then drop at 2.8B. The Multi-language LoRA configuration is the most extreme, shifting from negative at 160M to strongly positive at 1B (0.0975) and returning to negative at 2.8B (-0.0090). Java LoRA shows a similar reversal from positive at 1B (0.0798) to slightly negative at 2.8B (-0.0028).

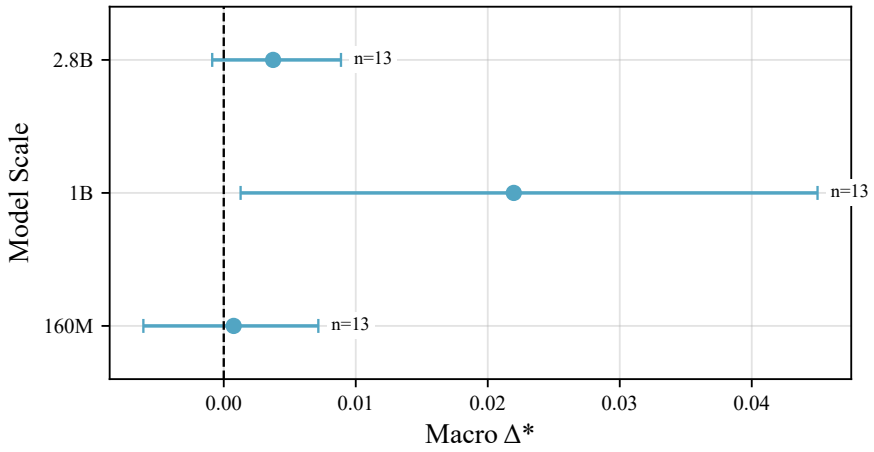

Figure 5.8: Forest plot of mean Δ^*_{macro} across matched triplets at each model scale, with 95% confidence intervals.

Figure 5.8 summarizes the matched-triplet results by reporting mean Δ^*_{macro} at each scale, with bootstrap confidence intervals reflecting variability across the 13 triplets.

The mean is close to zero at 160M and 2.8B, while it is higher at 1B; the bootstrap interval for 1B does not include zero, whereas the intervals for 160M and 2.8B do.

5.6 RQ5: Task-Specific Sensitivity

RQ5 asks which reasoning benchmarks exhibit the strongest CITE. We compare tasks by aggregating Δ^* over model–PEFT configurations (averaging code performance across the four code datasets for each configuration).

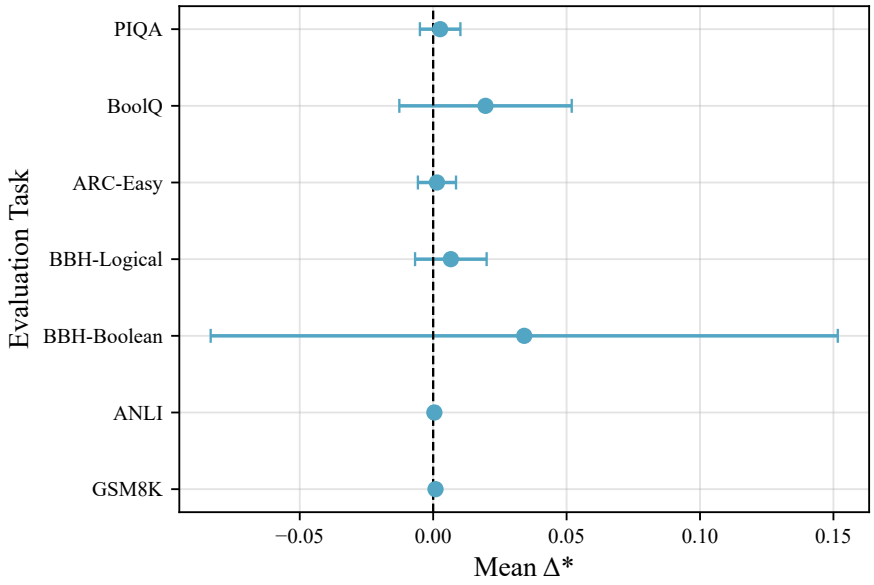

Figure 5.9: Forest plot of task-level mean Δ^*, with 95% confidence intervals.

Figure 5.9 shows mean Δ^* for each task. Each point is the mean Δ^* computed over the 8 model–PEFT configurations for that task, with whiskers indicating the 95% bootstrapped confidence interval.

BBH Boolean has the highest mean Δ^* (0.034), followed by BoolQ (0.020) and BBH Logical (0.007). The remaining tasks (ARC-Easy, PIQA, GSM8K, ANLI) have means between 0.001 and 0.003. All 95% confidence intervals include zero.

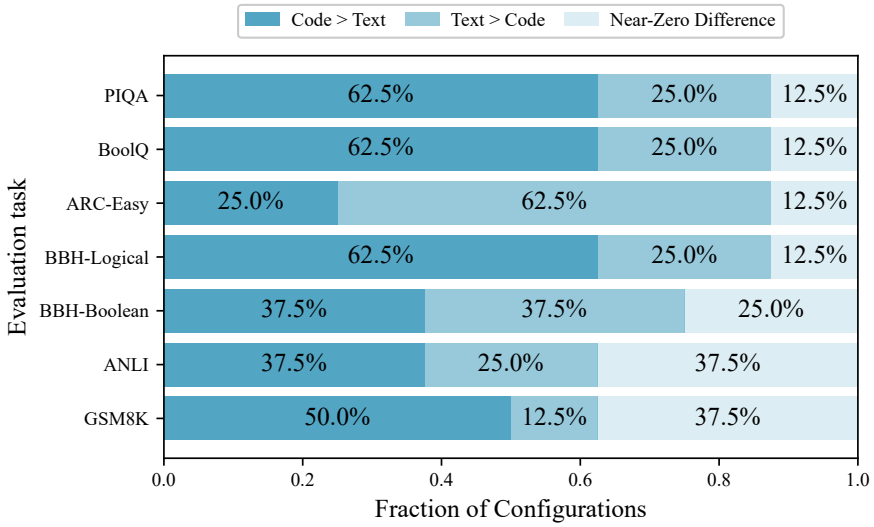

Figure 5.10: Stacked bar chart of Δ^* direction across tasks (near-zero difference: $|\Delta^*| \leq 0.001$).

Figure 5.10 complements the mean summary by showing how consistently each benchmark exhibits code advantage, text advantage, or near-zero differences across configurations.

- BBH Logical, BoolQ, and PIQA show the highest fractions of code-advantage cases.
- In contrast, ARC-Easy is dominated by text-advantage cases.
- BBH Boolean is more mixed, with comparable shares of code-advantage and text-advantage outcomes and a smaller near-zero fraction.
- GSM8K and ANLI contain a large near-zero fraction, consistent with their small mean effects.

A configuration-level breakdown of Δ^* values for each task is provided in Figure A.1.

6. Discussion

6.1 Overview

This chapter interprets the empirical findings from Chapter 5. The aim is to separate what is consistent from what is conditional, and to clarify what the results imply about the Code-Induced Transfer Effect (CITE).

The discussion follows the research questions in order, then closes with limitations and implications.

6.2 RQ1: Overall CITE

RQ1 asks whether code fine-tuning improves reasoning beyond a matched text fine-tuning baseline. Across all configurations, code outperforms text in 48.2% of cases, while text outperforms code in 30.4%, with the remaining cases near zero. This indicates that code exposure can help, but not reliably in every setting.

The heatmap in Figure 5.2 supports the same point. Some tasks sit close to zero across most configurations (for example GSM8K and ANLI), while others vary widely depending on the configuration (especially BBH Boolean, which includes both the strongest positive and strongest negative Δ^* values). This task-structured variation suggests that code fine-tuning transfers specific benefits rather than raising performance uniformly.

Takeaway
CITE appears as a targeted effect that depends on the task and configuration, not as a universal gain.

6.3 RQ2: Programming Language Comparison

RQ2 asks whether the programming language of the fine-tuning data changes CITE. Across the code datasets, the distributions of Δ^* are broadly similar. Java and Python behave almost identically: code has an advantage in 53.6% of configurations, text in 32.1%, and 14.3% are near zero. C++ and the multi-language subset show slightly fewer code-advantage cases (48.2%) and more near-zero outcomes, which suggests a weaker or less consistent separation from the text baseline.

The same picture appears in the distribution summaries. Medians are positive but small across all languages, and the means lie in a narrow range, from about 0.006 to 0.0176. The multi-language subset has the highest mean, but it also has the widest spread, which indicates higher variability rather than a uniformly stronger benefit. In every language subset, Δ^* includes both positive and negative values, so the direction of transfer is not determined by language alone.

A simple explanation for the multi-language behavior is diversity. Mixing languages exposes the model to varied syntax, libraries, and programming styles. This can increase the chance of learning structure that transfers, but it can also increase mismatch with specific evaluation tasks. With limited fine-tuning, this can lead to unstable results.

Overall, programming language does not appear to be the primary driver of CITE. If it were, we would expect clearer separation between the language distributions. Instead, they overlap heavily.

Takeaway
Language choice matters less than other factors, but Multi-language code behaves like a higher-variance intervention, with higher upside and higher downside.

6.4 RQ3: PEFT Methods Comparison

RQ3 asks whether the PEFT method changes CITE. PEFT choice is the clearest moderator in the study. When CITE is summarized at the configuration level using task-averaged macro Δ^*, the methods separate clearly.

LoRA yields the strongest CITE overall. It shows code advantage in 87.5% of configurations, with the highest mean (0.0355) and the highest median (0.0167). LoRA also has the widest spread, and its range crosses zero.

This pattern suggests that LoRA is high impact. It often turns code exposure into measurable gains, but it can also produce losses in some settings. In other words, LoRA amplifies the effect of the fine-tuning signal. Whether that effect helps or hurts depends on how well the code signal matches the evaluation task.

VeRA shows the opposite pattern. Text has the advantage in 87.5% of configurations, with a negative mean (-0.0071) and a negative median (-0.0043). Under this training setup, VeRA benefits more from natural-language fine-tuning than from code fine-tuning.

Prefix-Tuning is mildly positive on average (mean 0.0104, median 0.0037), and it includes a strong positive outlier (0.0620). In the heatmap of Figure 5.2, Prefix-Tuning shows especially large positives on BoolQ (0.094 at 160M, 0.083 at 1B). This suggests selective strength: Prefix-Tuning may not shift the model broadly, but it can help substantially when the task aligns with the kind of conditioning it provides.

IA3 sits very close to zero (mean -0.0012, median -0.0009) with a narrow range. This suggests that, under our setup, IA3 does not create a difference between code and text fine-tuning.

Takeaway

PEFT choice strongly shapes CITE: LoRA most often yields positive effects, VeRA tends to favor text, Prefix-Tuning shows selective wins, and IA3 mostly stays near zero.

6.5 RQ4: Model Scale Comparison

RQ4 asks how CITE changes with model size. The scale analysis uses matched triplets across Pythia-160M, Pythia-1B, and Pythia-2.8B, for a total of 13 matched triplets. Within this subset, CITE does not increase smoothly with model size.

The slope chart in Figure 5.7 shows different patterns by PEFT method:

- IA3 forms a consistent V-shape: slightly positive at 160M, negative at 1B, then close to the starting value at 2.8B.
- VeRA improves monotonically but stays negative across scales.
- Prefix-Tuning is mixed and depends on the dataset.
- LoRA shows the largest swings: it rises sharply from 160M to 1B, then drops at 2.8B.

A non-monotonic pattern is plausible because the fine-tuning setup is held fixed across scales. Larger models often require different step counts, learning rates, or adapter settings to reach a comparable level of adaptation. If these choices are not adjusted, the largest model may be under-tuned, or it may respond less predictably to the same update. In that case, the code-specific signal can look strongest at an intermediate scale, not because that scale is inherently best, but because it is where the fixed recipe happens to work most effectively.

The forest plot in Figure 5.8 supports this reading. The mean macro Δ^* is highest at 1B, and its bootstrap interval excludes zero, while the intervals for 160M and 2.8B include zero. This does not imply that 1B is best in general. It indicates that, under the current training and evaluation design, the code-specific benefit is most visible at 1B.

Takeaway

Within the matched subset, CITE is not monotonic in scale. Under a fixed recipe, the effect is clearest at 1B and weaker or less reliable at 160M and 2.8B.

6.6 RQ5: Task-Specific Sensitivity

RQ5 asks whether CITE depends on the evaluation task. Two aspects matter here. One is average effect size (mean Δ^*). The other is how consistently the effect points in one direction across configurations.

Across tasks, mean effects are small. BBH Boolean has the largest mean Δ^* (0.034), followed by BoolQ (0.020) and BBH Logical (0.007). The remaining tasks are very close to zero (roughly 0.001 to 0.003). Importantly, all task-level confidence intervals include zero. This means that, at the level of task averages and under this fine-tuning budget, no benchmark shows a clearly non-zero CITE.

At the same time, the direction plot shows that some tasks lean more often one way even when the mean is small. BoolQ, BBH Logical, and PIQA show the most frequent code advantage (62.5%). ARC-Easy shows the clearest tilt in the opposite direction, with text advantage in 62.5% of configurations. BBH Boolean is different: it is the most mixed task, with code and text advantage equally frequent (37.5% each) and a relatively large near-zero share (25%). GSM8K and ANLI also show high near-zero rates (37.5%), consistent with their near-zero means.

BBH Boolean stands out because it combines a positive mean with mixed directions. This implies that the outcome depends strongly on the specific model and PEFT configuration rather than showing a stable task-level trend. Figure A.1 supports this directly: BBH Boolean contains both the strongest positive and strongest negative Δ^* values in the study.

In practice, BBH Boolean behaves as a high-variance task: some configurations gain substantially from code fine-tuning, while others degrade.

ARC-Easy shows the clearest directional tilt toward the text baseline. Yet its mean remains close to zero, which suggests that the typical differences are small even when they more often favor text. This makes ARC-Easy a useful boundary case for the research question: in this setting, code fine-tuning does not transfer reliably to ARC-Easy.

For **GSM8K and ANLI**, the most noticeable feature is how often the outcome is effectively neutral (37.5%), together with means very close to zero. Across many configurations, code and text fine-tuning perform similarly on these tasks.

The safest conclusion is not that code can never help on math or inference, but that under the present fine-tuning budget and evaluation protocol, any code-specific advantage is small and not consistent at the task level.

A domain view: math, logic, commonsense

If we group tasks by domain, the overall signal is still modest and not uniform within any domain. In the logic domain, BBH Logical leans positive in direction, BBH Boolean shows the largest average gain but also the strongest dependence on configuration, and ANLI is mostly neutral. In the commonsense domain, BoolQ and PIQA lean positive in direction, while ARC-Easy leans negative. For math, GSM8K remains close to zero in this study.

Overall, these results do not suggest that any domain consistently benefits from code fine-tuning. Instead, the effect is mostly task-specific, even within the same domain.

Takeaway

Task sensitivity is real, but it appears more in direction and variability than in large average gains. The effect is mixed within each domain.

6.7 Limitations

This study is designed to isolate a code-specific transfer signal by comparing code fine-tuning against a matched text fine-tuning baseline. That makes the conclusions meaningful, but also places clear limits on what can be claimed.

First, many measured effects are very small. The neutrality threshold ($|\Delta^*| \leq 0.001$) is a practical way to treat tiny differences as practically negligible, but they could still represent genuine (albeit tiny) effects that we cannot reliably distinguish from noise. When results fall near zero, the safest statement is that no clear advantage is detected at the current resolution, rather than that no difference exists.

Second, the fine-tuning recipe is fixed across many settings. A fixed token budget, step count, and adapter configuration makes comparisons fair, but it also means that the recipe may not be equally suitable for every model size and every PEFT method. The non-monotonic scale results suggest that some models may be under-tuned or respond

less predictably under the same update. For this reason, the scale findings should be read as evidence about this training regime, not as a general rule that one size is best.

Third, the evaluation protocol is deliberately uniform. Using the same prompting and decoding settings across tasks improves comparability, but it can also hide effects that only appear under different prompting styles or decoding strategies. In other words, the thesis measures CITE under one consistent evaluation setting, and different settings could change the observed magnitudes.

Finally, the evidence is concentrated in one model family and two main sizes. Most experiments use Pythia-160M and Pythia-1B, while Pythia-2.8B is included only as a smaller matched probe. For this reason, the conclusions are strongest for the Pythia family and the tested size range.

Overall, these limitations do not weaken the main message of the thesis. They clarify that the results describe when CITE appears under a specific, controlled setup, and they highlight where follow-up experiments would be needed to test generality.

6.8 Implications

The results suggest that unsupervised code fine-tuning can benefit non-programming reasoning, but only under specific conditions. It is not a simple upgrade that applies everywhere, because the effect depends on choices that the model developer makes.

The clearest implication is that claims about code helping reasoning should be treated as method-dependent. Under the same data and training budget, different PEFT methods lead to very different outcomes. This means that positive or negative results cannot be attributed to code alone without considering how the model is updated.

A second implication is that transfer should not be assumed to generalize uniformly across tasks. Since some tasks benefit, some remain close to neutral, and others favor the text baseline, code fine-tuning should be validated on the target benchmark before it is adopted in practice. In this sense, code behaves less like a universal reasoning booster and more like a conditional source of transfer.

A third implication is that scaling does not guarantee stronger transfer under an unchanged recipe. The matched scale probe suggests that fine-tuning configurations may need to be adjusted as model size changes. As a result, negative or weak findings at

larger scale should not automatically be interpreted as evidence against CITE itself, but may instead reflect a mismatch between model size and adaptation setup.

Taken together, these implications point to a clear conclusion: unsupervised code fine-tuning should be treated as a conditional strategy whose impact depends on the adaptation method, the target task, and the training regime.

6.9 Summary

This chapter discussed the results through the five research questions and clarified what they imply about CITE. The main points are:

- CITE is present in the aggregate, but it is not uniform across configurations.
- Programming language choice has a modest effect, while multi-language code shows higher variability.
- PEFT method is the strongest moderator: LoRA most often yields positive effects, VeRA tends to favor text, Prefix-Tuning shows selective gains, and IA3 stays close to zero.
- CITE is not monotonic in model size under a fixed recipe; the clearest average signal appears at 1B in the matched probe.
- Task sensitivity is pronounced and does not align cleanly by domain. Within the logic domain, BBH Logical leans positive, ANLI is near neutral, and BBH Boolean is highly configuration-dependent. Within the commonsense domain, BoolQ and PIQA lean positive while ARC-Easy most consistently favors text. GSM8K remains close to zero.

Overall, the central conclusion is that CITE is real but conditional. Unsupervised code fine-tuning can transfer to non-programming reasoning tasks, but the effect depends on how the model is adapted and what it is evaluated on.

7. Conclusion

7.1 Overview

This thesis studied whether unsupervised code fine-tuning can improve reasoning performance in non-programming domains. To isolate a code-specific effect, it compared code fine-tuning against a matched natural-language fine-tuning baseline using the same model family, PEFT method, and training budget. The difference was measured using Δ^*, which captures how much more (or less) code helps relative to text.

The results show that CITE is real but conditional. Code fine-tuning can transfer to some reasoning tasks, but the effect depends on the adaptation method, the target task, and the training regime.

7.2 Answers to the research questions

The five research questions can be answered as follows.

RQ1: A code-specific transfer effect exists, but it is not uniform. Across the full set of configurations, code outperforms text in many cases, yet text also wins in a substantial fraction, and near-zero outcomes are common. This means that code exposure can help, but it should not be treated as a guaranteed improvement.

RQ2: The programming language of the code data is not a primary driver of CITE. Java, Python, and C++ show broadly similar outcomes, with only small differences in average behavior. Multi-language code shows higher variability, with both higher upside and higher downside, rather than a consistently stronger benefit.

RQ3: The PEFT method is the strongest moderator in this study. LoRA is most often associated with positive CITE and also shows the largest swings, while VeRA tends to favor the text baseline. Prefix-Tuning can produce strong gains in specific cases, and IA3 remains close to neutral. This shows that CITE is not only a property of the fine-tuning data, but also of how the update is applied.

RQ4: CITE is not monotonic in model size under a fixed fine-tuning recipe. In the matched scale probe, the clearest average signal appears at 1B, while 160M and 2.8B are weaker or less reliable. This suggests that scaling can require changes to the fine-tuning setup if the goal is to preserve or amplify transfer.

RQ5: CITE is task-sensitive, and the results do not align cleanly by domain. Within logical reasoning, BBH Logical leans positive in direction, ANLI is near neutral, and BBH Boolean is highly configuration-dependent, with both strong gains and strong losses. Within commonsense reasoning, BoolQ and PIQA lean positive while ARC-Easy most consistently favors the text baseline. For mathematical reasoning, GSM8K remains close to zero in this setup. Overall, transfer appears specific and conditional, rather than a uniform improvement across domains.

7.3 Contributions

This thesis makes three main contributions.

First, it provides a clear operational definition of CITE using Δ^*, which measures code-specific transfer by subtracting a matched text fine-tuning baseline. This framing helps separate the effect of code from the effect of fine-tuning in general.

Second, it offers a controlled empirical comparison across multiple factors that are often mixed together in informal claims about code and reasoning. In particular, it compares programming languages, PEFT methods, model sizes, and task types under a consistent training and evaluation protocol.

Third, it reports concrete patterns that can guide future work. The most important patterns are the strong role of PEFT method, the non-monotonic scale behavior under a fixed recipe, and the clear task sensitivity of the effect.

7.4 Limitations

The conclusions of this thesis are tied to the chosen setup. Many measured effects are small, and near-zero values should be read as no clear difference detected at the current resolution. The fine-tuning recipe is held fixed across settings, which improves fairness but may not be optimal for every model size and adaptation method. The evaluation

protocol is also fixed, which supports comparability but may hide effects that appear under different prompting or decoding strategies. Finally, most evidence comes from the Pythia family at 160M and 1B, with 2.8B included only as a smaller matched probe, so generalization beyond this scope should be treated cautiously.

7.5 Future work

Several follow-up directions could strengthen and extend these findings.

A first direction is to scale the fine-tuning recipe with model size. This includes exploring different step counts, learning rates, and adapter settings, especially for larger models where non-monotonic effects appear under a fixed recipe.

A second direction is to vary evaluation settings. Different prompting strategies or decoding methods could change how reasoning is expressed, which may change the apparent size of CITE on tasks that currently look neutral.

A third direction is to add more benchmarks within each domain, to test whether the effects remain mostly task-specific or become clearer at the domain level.

Finally, targeted ablations could explain why PEFT methods behave differently. For example, varying adapter capacity within a method could help determine whether the differences are driven by expressivity, stability, or other factors.

7.6 Closing remarks

The main conclusion of this thesis is simple. Unsupervised code fine-tuning can enhance reasoning in non-programming domains, but it behaves like a conditional tool rather than a general solution. It helps in some settings, does little in others, and can harm performance when misaligned. Understanding and using CITE therefore requires attention to the adaptation method, the target task, and the training regime.

Bibliography

[1] Viraat Aryabumi, Yixuan Su, Raymond Ma, Adrien Morisot, Ivan Zhang, Acyr Locatelli, Marzieh Fadaee, Ahmet Üstün, and Sara Hooker. *To Code, or Not To Code? Exploring Impact of Code in Pre-training*. 2024. arXiv: 2408.10914 [cs.CL]. URL: https://arxiv.org/abs/2408.10914.

[2] Bryan Li, Tamer Alkhouli, Daniele Bonadiman, Nikolaos Pappas, and Saab Mansour. "Eliciting Better Multilingual Structured Reasoning from LLMs through Code". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 5154–5169. DOI: 10.18653/v1/2024.acl-long.281. URL: https://aclanthology.org/2024.acl-long.281/.

[3] Xinlu Zhang, Zhiyu Zoey Chen, Xi Ye, Xianjun Yang, Lichang Chen, William Yang Wang, and Linda Ruth Petzold. "Unveiling the impact of coding data instruction fine-tuning on large language models reasoning". In: *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*. AAAI'25/IAAI'25/EAAI'25. AAAI Press, 2025. ISBN: 978-1-57735-897-8. DOI: 10.1609/aaai.v39i24.34789. URL: https://doi.org/10.1609/aaai.v39i24.34789.

[4] Dayu Yang, Tianyang Liu, Daoan Zhang, Antoine Simoulin, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, Xin Qian, Grey Yang, Jiebo Luo, and Julian McAuley. "Code to Think, Think to Code: A Survey on Code-Enhanced Reasoning and Reasoning-Driven Code Intelligence in LLMs". In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Ed. by Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 2586–2616. ISBN: 979-8-89176-332-6. DOI: 10.18653/v1/2025.emnlp-main.130. URL: https://aclanthology.org/2025.emnlp-main.130/.

[5] Maggie Ziyu Huan, Yuetai Li, Tuney Zheng, XiaoYu Xu, Seungone Kim, Minxin Du, Radha Poovendran, Graham Neubig, and Xiang Yue. "Does Math Reasoning Improve General LLM Capabilities? Understanding Transferability of

LLM Reasoning". In: *ArXiv* abs/2507.00432 (2025). URL: https://api.semanticscholar.org/CorpusID:280146966.

[6] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. "Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling". In: *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*. Ed. by Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 2397–2430. URL: https://proceedings.mlr.press/v202/biderman23a.html.

[7] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. "The Pile: An 800GB Dataset of Diverse Text for Language Modeling". In: *arXiv preprint arXiv:2101.00027* (2020).

[8] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. "Emergent Abilities of Large Language Models". In: *Transactions on Machine Learning Research* (2022). Survey Certification. ISSN: 2835-8856. URL: https://openreview.net/forum?id=yzkSU5zdwD.

[9] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin A Raffel. "Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning". In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh. Vol. 35. Curran Associates, Inc., 2022, pp. 1950–1965. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/0cde695b83bd186c1fd456302888454c-Paper-Conference.pdf.

[10] Dawid Jan Kopiczko, Tijmen Blankevoort, and Yuki M Asano. "VeRA: Vector-based Random Matrix Adaptation". In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=NjNfLdxr3A.

[11] Xiang Lisa Li and Percy Liang. "Prefix-Tuning: Optimizing Continuous Prompts for Generation". In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on*

Natural Language Processing (Volume 1: Long Papers). Ed. by Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli. Online: Association for Computational Linguistics, Aug. 2021, pp. 4582–4597. DOI: 10.18653/v1/2021.acl-long.353. URL: https://aclanthology.org/2021.acl-long.353/.

[12] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9.

[13] Raymond Li, Loubna Ben allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia LI, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Joel Lamy-Poirier, Joao Monteiro, Nicolas Gontier, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Ben Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason T Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Urvashi Bhattacharyya, Wenhao Yu, Sasha Luccioni, Paulo Villegas, Fedor Zhdanov, Tony Lee, Nadav Timor, Jennifer Ding, Claire S Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro Von Werra, and Harm de Vries. "StarCoder: may the source be with you!" In: *Transactions on Machine Learning Research* (2023). Reproducibility Certification. ISSN: 2835-8856. URL: https://openreview.net/forum?id=KoFOg41haE.

[14] BigCode Project. *bigcode/starcoderdata: StarCoder Training Dataset*. Hugging Face Datasets. Accessed: February 2026. 2023. URL: https://huggingface.co/datasets/bigcode/starcoderdata.

[15] Denis Kocetkov, Raymond Li, Loubna Ben allal, Jia LI, Chenghao Mou, Yacine Jernite, Margaret Mitchell, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro Von Werra, and Harm de Vries. "The Stack: 3 TB of permissively licensed source code". In: *Transactions on Machine Learning Research* (2023). ISSN: 2835-8856. URL: https://openreview.net/forum?id=pxpbTdUEpD.

[16] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. "The FineWeb datasets: decanting the web for the finest text data at scale". In: *Proceedings of the 38th International Conference on Neural Information Processing*

Systems. NIPS '24. Vancouver, BC, Canada: Curran Associates Inc., 2024. ISBN: 9798331314385.

[17] HuggingFaceFW. *HuggingFaceFW/fineweb-edu: FineWeb-Edu – the Finest Collection of Educational Content*. Hugging Face Datasets. Accessed: February 2026. 2024. URL: https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu.

[18] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. "Training Verifiers to Solve Math Word Problems". In: *CoRR* abs/2110.14168 (2021). arXiv: 2110.14168. URL: https://arxiv.org/abs/2110.14168.

[19] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. "Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them". In: *Findings of the Association for Computational Linguistics: ACL 2023*. Ed. by Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 13003–13051. DOI: 10.18653/v1/2023.findings-acl.824. URL: https://aclanthology.org/2023.findings-acl.824/.

[20] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. "Adversarial NLI: A New Benchmark for Natural Language Understanding". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Ed. by Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault. Online: Association for Computational Linguistics, July 2020, pp. 4885–4901. DOI: 10.18653/v1/2020.acl-main.441. URL: https://aclanthology.org/2020.acl-main.441/.

[21] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge". In: *ArXiv* abs/1803.05457 (2018). URL: https://api.semanticscholar.org/CorpusID:3922816.

[22] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for

Computational Linguistics, June 2019, pp. 2924–2936. DOI: 10.18653/v1/N19-1300. URL: https://aclanthology.org/N19-1300/.

[23] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. "PIQA: Reasoning about Physical Commonsense in Natural Language". In: *AAAI Conference on Artificial Intelligence*. 2019. URL: https://api.semanticscholar.org/CorpusID:208290939.

[24] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. "PyTorch: an imperative style, high-performance deep learning library". In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[25] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. "Transformers: State-of-the-Art Natural Language Processing". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Ed. by Qun Liu and David Schlangen. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. DOI: 10.18653/v1/2020.emnlp-demos.6. URL: https://aclanthology.org/2020.emnlp-demos.6/.

[26] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods*. https://github.com/huggingface/peft. 2022.

[27] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. *TRL: Transformers Reinforcement Learning*. https://github.com/huggingface/trl. 2020. URL: https://github.com/huggingface/trl.

[28] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang,

Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. *The Language Model Evaluation Harness*. Version v0.4.3. July 2024. DOI: 10.5281/zenodo.12608602. URL: https://zenodo.org/records/12608602.

[29] Omry Yadan. *Hydra - A framework for elegantly configuring complex applications*. Github. 2019. URL: https://github.com/facebookresearch/hydra.

A. Appendix

A.1 Raw Evaluation Results

Table A.1: Raw evaluation results for all baseline and fine-tuned models across reasoning benchmarks.

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
baseline	pythia-160m	none	none	anli	0.3355	0.0145
baseline	pythia-160m	none	none	arc_easy	0.4474	0.0102
baseline	pythia-160m	none	none	bbh_boolean	0.4880	0.0317
baseline	pythia-160m	none	none	bbh_logical	0.0480	0.0135
baseline	pythia-160m	none	none	boolq	0.5235	0.0087
baseline	pythia-160m	none	none	gsm8k	0.0220	0.0040
baseline	pythia-160m	none	none	piqa	0.6137	0.0114
baseline	pythia-1b	none	none	anli	0.3237	0.0144
baseline	pythia-1b	none	none	arc_easy	0.5909	0.0101
baseline	pythia-1b	none	none	bbh_boolean	0.4440	0.0315
baseline	pythia-1b	none	none	bbh_logical	0.2840	0.0286
baseline	pythia-1b	none	none	boolq	0.5869	0.0086
baseline	pythia-1b	none	none	gsm8k	0.0220	0.0040
baseline	pythia-1b	none	none	piqa	0.7122	0.0106
baseline	pythia-2.8b	none	none	anli	0.3403	0.0146
baseline	pythia-2.8b	none	none	arc_easy	0.6742	0.0096
baseline	pythia-2.8b	none	none	bbh_boolean	0.4480	0.0315
baseline	pythia-2.8b	none	none	bbh_logical	0.3600	0.0304
baseline	pythia-2.8b	none	none	boolq	0.6703	0.0082
baseline	pythia-2.8b	none	none	gsm8k	0.0273	0.0045
baseline	pythia-2.8b	none	none	piqa	0.7465	0.0102
finetuned	pythia-160m	ia3	fineweb-edu-sample-10BT	anli	0.3386	0.0145
finetuned	pythia-160m	ia3	fineweb-edu-sample-10BT	arc_easy	0.4390	0.0102
finetuned	pythia-160m	ia3	fineweb-edu-sample-10BT	bbh_boolean	0.4880	0.0317
finetuned	pythia-160m	ia3	fineweb-edu-sample-10BT	bbh_logical	0.0960	0.0187
finetuned	pythia-160m	ia3	fineweb-edu-sample-10BT	boolq	0.5122	0.0087
finetuned	pythia-160m	ia3	fineweb-edu-sample-10BT	gsm8k	0.0205	0.0039
finetuned	pythia-160m	ia3	fineweb-edu-sample-10BT	piqa	0.6192	0.0113
finetuned	pythia-160m	ia3	starcoderdata-cpp	anli	0.3358	0.0145
finetuned	pythia-160m	ia3	starcoderdata-cpp	arc_easy	0.4419	0.0102
finetuned	pythia-160m	ia3	starcoderdata-cpp	bbh_boolean	0.5040	0.0317
finetuned	pythia-160m	ia3	starcoderdata-cpp	bbh_logical	0.0640	0.0155
finetuned	pythia-160m	ia3	starcoderdata-cpp	boolq	0.5235	0.0087
finetuned	pythia-160m	ia3	starcoderdata-cpp	gsm8k	0.0182	0.0037
finetuned	pythia-160m	ia3	starcoderdata-cpp	piqa	0.6240	0.0113
finetuned	pythia-160m	ia3	starcoderdata-java	anli	0.3399	0.0146
finetuned	pythia-160m	ia3	starcoderdata-java	arc_easy	0.4449	0.0102
finetuned	pythia-160m	ia3	starcoderdata-java	bbh_boolean	0.5160	0.0317
finetuned	pythia-160m	ia3	starcoderdata-java	bbh_logical	0.0920	0.0183
finetuned	pythia-160m	ia3	starcoderdata-java	boolq	0.5193	0.0087
finetuned	pythia-160m	ia3	starcoderdata-java	gsm8k	0.0182	0.0037
finetuned	pythia-160m	ia3	starcoderdata-java	piqa	0.6224	0.0113

Continued on next page

Continued from previous page

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
finetuned	pythia-160m	ia3	starcoderdata-multi	anli	0.3404	0.0146
finetuned	pythia-160m	ia3	starcoderdata-multi	arc_easy	0.4428	0.0102
finetuned	pythia-160m	ia3	starcoderdata-multi	bbh_boolean	0.5400	0.0316
finetuned	pythia-160m	ia3	starcoderdata-multi	bbh_logical	0.0640	0.0155
finetuned	pythia-160m	ia3	starcoderdata-multi	boolq	0.5217	0.0087
finetuned	pythia-160m	ia3	starcoderdata-multi	gsm8k	0.0197	0.0038
finetuned	pythia-160m	ia3	starcoderdata-multi	piqa	0.6143	0.0114
finetuned	pythia-160m	ia3	starcoderdata-python	anli	0.3426	0.0146
finetuned	pythia-160m	ia3	starcoderdata-python	arc_easy	0.4474	0.0102
finetuned	pythia-160m	ia3	starcoderdata-python	bbh_boolean	0.5400	0.0316
finetuned	pythia-160m	ia3	starcoderdata-python	bbh_logical	0.0880	0.0180
finetuned	pythia-160m	ia3	starcoderdata-python	boolq	0.5187	0.0087
finetuned	pythia-160m	ia3	starcoderdata-python	gsm8k	0.0174	0.0036
finetuned	pythia-160m	ia3	starcoderdata-python	piqa	0.6246	0.0113
finetuned	pythia-160m	lora	fineweb-edu-sample-10BT	anli	0.3419	0.0146
finetuned	pythia-160m	lora	fineweb-edu-sample-10BT	arc_easy	0.3434	0.0097
finetuned	pythia-160m	lora	fineweb-edu-sample-10BT	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	lora	fineweb-edu-sample-10BT	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	lora	fineweb-edu-sample-10BT	boolq	0.6076	0.0085
finetuned	pythia-160m	lora	fineweb-edu-sample-10BT	gsm8k	0.0045	0.0019
finetuned	pythia-160m	lora	fineweb-edu-sample-10BT	piqa	0.5686	0.0116
finetuned	pythia-160m	lora	starcoderdata-cpp	anli	0.3403	0.0146
finetuned	pythia-160m	lora	starcoderdata-cpp	arc_easy	0.3889	0.0100
finetuned	pythia-160m	lora	starcoderdata-cpp	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	lora	starcoderdata-cpp	bbh_logical	0.0360	0.0118
finetuned	pythia-160m	lora	starcoderdata-cpp	boolq	0.5768	0.0086
finetuned	pythia-160m	lora	starcoderdata-cpp	gsm8k	0.0053	0.0020
finetuned	pythia-160m	lora	starcoderdata-cpp	piqa	0.5909	0.0115
finetuned	pythia-160m	lora	starcoderdata-java	anli	0.3387	0.0145
finetuned	pythia-160m	lora	starcoderdata-java	arc_easy	0.3771	0.0099
finetuned	pythia-160m	lora	starcoderdata-java	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	lora	starcoderdata-java	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	lora	starcoderdata-java	boolq	0.5606	0.0087
finetuned	pythia-160m	lora	starcoderdata-java	gsm8k	0.0114	0.0029
finetuned	pythia-160m	lora	starcoderdata-java	piqa	0.5903	0.0115
finetuned	pythia-160m	lora	starcoderdata-multi	anli	0.3461	0.0146
finetuned	pythia-160m	lora	starcoderdata-multi	arc_easy	0.3413	0.0097
finetuned	pythia-160m	lora	starcoderdata-multi	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	lora	starcoderdata-multi	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	lora	starcoderdata-multi	boolq	0.5104	0.0087
finetuned	pythia-160m	lora	starcoderdata-multi	gsm8k	0.0061	0.0021
finetuned	pythia-160m	lora	starcoderdata-multi	piqa	0.5816	0.0115
finetuned	pythia-160m	lora	starcoderdata-python	anli	0.3383	0.0145
finetuned	pythia-160m	lora	starcoderdata-python	arc_easy	0.3674	0.0099
finetuned	pythia-160m	lora	starcoderdata-python	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	lora	starcoderdata-python	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	lora	starcoderdata-python	boolq	0.5969	0.0086
finetuned	pythia-160m	lora	starcoderdata-python	gsm8k	0.0068	0.0023
finetuned	pythia-160m	lora	starcoderdata-python	piqa	0.5822	0.0115
finetuned	pythia-160m	prefix-tuning	fineweb-edu-sample-10BT	anli	0.3343	0.0145
finetuned	pythia-160m	prefix-tuning	fineweb-edu-sample-10BT	arc_easy	0.3001	0.0094
finetuned	pythia-160m	prefix-tuning	fineweb-edu-sample-10BT	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	fineweb-edu-sample-10BT	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	fineweb-edu-sample-10BT	boolq	0.3838	0.0085
finetuned	pythia-160m	prefix-tuning	fineweb-edu-sample-10BT	gsm8k	0.0008	0.0008
finetuned	pythia-160m	prefix-tuning	fineweb-edu-sample-10BT	piqa	0.5800	0.0115

Continued on next page

Continued from previous page

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
finetuned	pythia-160m	prefix-tuning	starcoderdata-cpp	anli	0.3419	0.0146
finetuned	pythia-160m	prefix-tuning	starcoderdata-cpp	arc_easy	0.2992	0.0094
finetuned	pythia-160m	prefix-tuning	starcoderdata-cpp	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-cpp	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-cpp	boolq	0.4437	0.0087
finetuned	pythia-160m	prefix-tuning	starcoderdata-cpp	gsm8k	0.0023	0.0013
finetuned	pythia-160m	prefix-tuning	starcoderdata-cpp	piqa	0.5539	0.0116
finetuned	pythia-160m	prefix-tuning	starcoderdata-java	anli	0.3402	0.0146
finetuned	pythia-160m	prefix-tuning	starcoderdata-java	arc_easy	0.2828	0.0092
finetuned	pythia-160m	prefix-tuning	starcoderdata-java	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-java	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-java	boolq	0.4153	0.0086
finetuned	pythia-160m	prefix-tuning	starcoderdata-java	gsm8k	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-java	piqa	0.5620	0.0116
finetuned	pythia-160m	prefix-tuning	starcoderdata-multi	anli	0.3293	0.0144
finetuned	pythia-160m	prefix-tuning	starcoderdata-multi	arc_easy	0.3077	0.0095
finetuned	pythia-160m	prefix-tuning	starcoderdata-multi	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-multi	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-multi	boolq	0.4413	0.0087
finetuned	pythia-160m	prefix-tuning	starcoderdata-multi	gsm8k	0.0023	0.0013
finetuned	pythia-160m	prefix-tuning	starcoderdata-multi	piqa	0.5713	0.0115
finetuned	pythia-160m	prefix-tuning	starcoderdata-python	anli	0.3405	0.0146
finetuned	pythia-160m	prefix-tuning	starcoderdata-python	arc_easy	0.2845	0.0093
finetuned	pythia-160m	prefix-tuning	starcoderdata-python	bbh_boolean	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-python	bbh_logical	0.0000	0.0000
finetuned	pythia-160m	prefix-tuning	starcoderdata-python	boolq	0.6119	0.0085
finetuned	pythia-160m	prefix-tuning	starcoderdata-python	gsm8k	0.0023	0.0013
finetuned	pythia-160m	prefix-tuning	starcoderdata-python	piqa	0.5571	0.0116
finetuned	pythia-160m	vera	fineweb-edu-sample-10BT	anli	0.3379	0.0145
finetuned	pythia-160m	vera	fineweb-edu-sample-10BT	arc_easy	0.4457	0.0102
finetuned	pythia-160m	vera	fineweb-edu-sample-10BT	bbh_boolean	0.5160	0.0317
finetuned	pythia-160m	vera	fineweb-edu-sample-10BT	bbh_logical	0.0440	0.0130
finetuned	pythia-160m	vera	fineweb-edu-sample-10BT	boolq	0.5092	0.0087
finetuned	pythia-160m	vera	fineweb-edu-sample-10BT	gsm8k	0.0197	0.0038
finetuned	pythia-160m	vera	fineweb-edu-sample-10BT	piqa	0.6099	0.0114
finetuned	pythia-160m	vera	starcoderdata-cpp	anli	0.3395	0.0145
finetuned	pythia-160m	vera	starcoderdata-cpp	arc_easy	0.4478	0.0102
finetuned	pythia-160m	vera	starcoderdata-cpp	bbh_boolean	0.3880	0.0309
finetuned	pythia-160m	vera	starcoderdata-cpp	bbh_logical	0.0480	0.0135
finetuned	pythia-160m	vera	starcoderdata-cpp	boolq	0.5251	0.0087
finetuned	pythia-160m	vera	starcoderdata-cpp	gsm8k	0.0182	0.0037
finetuned	pythia-160m	vera	starcoderdata-cpp	piqa	0.6137	0.0114
finetuned	pythia-160m	vera	starcoderdata-java	anli	0.3369	0.0145
finetuned	pythia-160m	vera	starcoderdata-java	arc_easy	0.4386	0.0102
finetuned	pythia-160m	vera	starcoderdata-java	bbh_boolean	0.3400	0.0300
finetuned	pythia-160m	vera	starcoderdata-java	bbh_logical	0.0480	0.0135
finetuned	pythia-160m	vera	starcoderdata-java	boolq	0.5174	0.0087
finetuned	pythia-160m	vera	starcoderdata-java	gsm8k	0.0174	0.0036
finetuned	pythia-160m	vera	starcoderdata-java	piqa	0.6257	0.0113
finetuned	pythia-160m	vera	starcoderdata-multi	anli	0.3400	0.0146
finetuned	pythia-160m	vera	starcoderdata-multi	arc_easy	0.4457	0.0102
finetuned	pythia-160m	vera	starcoderdata-multi	bbh_boolean	0.3880	0.0309
finetuned	pythia-160m	vera	starcoderdata-multi	bbh_logical	0.0920	0.0183
finetuned	pythia-160m	vera	starcoderdata-multi	boolq	0.5232	0.0087
finetuned	pythia-160m	vera	starcoderdata-multi	gsm8k	0.0205	0.0039
finetuned	pythia-160m	vera	starcoderdata-multi	piqa	0.6137	0.0114

Continued on next page

Continued from previous page

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
finetuned	pythia-160m	vera	starcoderdata-python	anli	0.3346	0.0145
finetuned	pythia-160m	vera	starcoderdata-python	arc_easy	0.4461	0.0102
finetuned	pythia-160m	vera	starcoderdata-python	bbh_boolean	0.4160	0.0312
finetuned	pythia-160m	vera	starcoderdata-python	bbh_logical	0.0800	0.0172
finetuned	pythia-160m	vera	starcoderdata-python	boolq	0.5153	0.0087
finetuned	pythia-160m	vera	starcoderdata-python	gsm8k	0.0235	0.0042
finetuned	pythia-160m	vera	starcoderdata-python	piqa	0.6262	0.0113
finetuned	pythia-1b	ia3	fineweb-edu-sample-10BT	anli	0.3248	0.0144
finetuned	pythia-1b	ia3	fineweb-edu-sample-10BT	arc_easy	0.5922	0.0101
finetuned	pythia-1b	ia3	fineweb-edu-sample-10BT	bbh_boolean	0.5480	0.0315
finetuned	pythia-1b	ia3	fineweb-edu-sample-10BT	bbh_logical	0.3200	0.0296
finetuned	pythia-1b	ia3	fineweb-edu-sample-10BT	boolq	0.5887	0.0086
finetuned	pythia-1b	ia3	fineweb-edu-sample-10BT	gsm8k	0.0167	0.0035
finetuned	pythia-1b	ia3	fineweb-edu-sample-10BT	piqa	0.7122	0.0106
finetuned	pythia-1b	ia3	starcoderdata-cpp	anli	0.3241	0.0144
finetuned	pythia-1b	ia3	starcoderdata-cpp	arc_easy	0.5829	0.0101
finetuned	pythia-1b	ia3	starcoderdata-cpp	bbh_boolean	0.4720	0.0316
finetuned	pythia-1b	ia3	starcoderdata-cpp	bbh_logical	0.3480	0.0302
finetuned	pythia-1b	ia3	starcoderdata-cpp	boolq	0.5869	0.0086
finetuned	pythia-1b	ia3	starcoderdata-cpp	gsm8k	0.0167	0.0035
finetuned	pythia-1b	ia3	starcoderdata-cpp	piqa	0.7133	0.0106
finetuned	pythia-1b	ia3	starcoderdata-java	anli	0.3277	0.0144
finetuned	pythia-1b	ia3	starcoderdata-java	arc_easy	0.5867	0.0101
finetuned	pythia-1b	ia3	starcoderdata-java	bbh_boolean	0.4520	0.0315
finetuned	pythia-1b	ia3	starcoderdata-java	bbh_logical	0.3400	0.0300
finetuned	pythia-1b	ia3	starcoderdata-java	boolq	0.5902	0.0086
finetuned	pythia-1b	ia3	starcoderdata-java	gsm8k	0.0205	0.0039
finetuned	pythia-1b	ia3	starcoderdata-java	piqa	0.7133	0.0106
finetuned	pythia-1b	ia3	starcoderdata-multi	anli	0.3223	0.0144
finetuned	pythia-1b	ia3	starcoderdata-multi	arc_easy	0.5901	0.0101
finetuned	pythia-1b	ia3	starcoderdata-multi	bbh_boolean	0.4920	0.0317
finetuned	pythia-1b	ia3	starcoderdata-multi	bbh_logical	0.3200	0.0296
finetuned	pythia-1b	ia3	starcoderdata-multi	boolq	0.5872	0.0086
finetuned	pythia-1b	ia3	starcoderdata-multi	gsm8k	0.0182	0.0037
finetuned	pythia-1b	ia3	starcoderdata-multi	piqa	0.7144	0.0105
finetuned	pythia-1b	ia3	starcoderdata-python	anli	0.3287	0.0144
finetuned	pythia-1b	ia3	starcoderdata-python	arc_easy	0.5875	0.0101
finetuned	pythia-1b	ia3	starcoderdata-python	bbh_boolean	0.5120	0.0317
finetuned	pythia-1b	ia3	starcoderdata-python	bbh_logical	0.3440	0.0301
finetuned	pythia-1b	ia3	starcoderdata-python	boolq	0.5881	0.0086
finetuned	pythia-1b	ia3	starcoderdata-python	gsm8k	0.0182	0.0037
finetuned	pythia-1b	ia3	starcoderdata-python	piqa	0.7133	0.0106
finetuned	pythia-1b	lora	fineweb-edu-sample-10BT	anli	0.3305	0.0144
finetuned	pythia-1b	lora	fineweb-edu-sample-10BT	arc_easy	0.5762	0.0101
finetuned	pythia-1b	lora	fineweb-edu-sample-10BT	bbh_boolean	0.0040	0.0040
finetuned	pythia-1b	lora	fineweb-edu-sample-10BT	bbh_logical	0.0000	0.0000
finetuned	pythia-1b	lora	fineweb-edu-sample-10BT	boolq	0.6006	0.0086
finetuned	pythia-1b	lora	fineweb-edu-sample-10BT	gsm8k	0.0144	0.0033
finetuned	pythia-1b	lora	fineweb-edu-sample-10BT	piqa	0.6975	0.0107
finetuned	pythia-1b	lora	starcoderdata-cpp	anli	0.3315	0.0145
finetuned	pythia-1b	lora	starcoderdata-cpp	arc_easy	0.5715	0.0102
finetuned	pythia-1b	lora	starcoderdata-cpp	bbh_boolean	0.5320	0.0316
finetuned	pythia-1b	lora	starcoderdata-cpp	bbh_logical	0.0120	0.0069
finetuned	pythia-1b	lora	starcoderdata-cpp	boolq	0.6171	0.0085
finetuned	pythia-1b	lora	starcoderdata-cpp	gsm8k	0.0136	0.0032
finetuned	pythia-1b	lora	starcoderdata-cpp	piqa	0.6997	0.0107

Continued on next page

Continued from previous page

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
finetuned	pythia-1b	lora	starcoderdata-java	anli	0.3335	0.0145
finetuned	pythia-1b	lora	starcoderdata-java	arc_easy	0.5791	0.0101
finetuned	pythia-1b	lora	starcoderdata-java	bbh_boolean	0.5240	0.0316
finetuned	pythia-1b	lora	starcoderdata-java	bbh_logical	0.0040	0.0040
finetuned	pythia-1b	lora	starcoderdata-java	boolq	0.6110	0.0085
finetuned	pythia-1b	lora	starcoderdata-java	gsm8k	0.0159	0.0034
finetuned	pythia-1b	lora	starcoderdata-java	piqa	0.7144	0.0105
finetuned	pythia-1b	lora	starcoderdata-multi	anli	0.3321	0.0145
finetuned	pythia-1b	lora	starcoderdata-multi	arc_easy	0.5737	0.0101
finetuned	pythia-1b	lora	starcoderdata-multi	bbh_boolean	0.5480	0.0315
finetuned	pythia-1b	lora	starcoderdata-multi	bbh_logical	0.1080	0.0197
finetuned	pythia-1b	lora	starcoderdata-multi	boolq	0.6116	0.0085
finetuned	pythia-1b	lora	starcoderdata-multi	gsm8k	0.0212	0.0040
finetuned	pythia-1b	lora	starcoderdata-multi	piqa	0.7111	0.0106
finetuned	pythia-1b	lora	starcoderdata-python	anli	0.3295	0.0144
finetuned	pythia-1b	lora	starcoderdata-python	arc_easy	0.5812	0.0101
finetuned	pythia-1b	lora	starcoderdata-python	bbh_boolean	0.1400	0.0220
finetuned	pythia-1b	lora	starcoderdata-python	bbh_logical	0.0120	0.0069
finetuned	pythia-1b	lora	starcoderdata-python	boolq	0.5969	0.0086
finetuned	pythia-1b	lora	starcoderdata-python	gsm8k	0.0174	0.0036
finetuned	pythia-1b	lora	starcoderdata-python	piqa	0.7078	0.0106
finetuned	pythia-1b	prefix-tuning	fineweb-edu-sample-10BT	anli	0.3243	0.0144
finetuned	pythia-1b	prefix-tuning	fineweb-edu-sample-10BT	arc_easy	0.5539	0.0102
finetuned	pythia-1b	prefix-tuning	fineweb-edu-sample-10BT	bbh_boolean	0.1840	0.0246
finetuned	pythia-1b	prefix-tuning	fineweb-edu-sample-10BT	bbh_logical	0.2640	0.0279
finetuned	pythia-1b	prefix-tuning	fineweb-edu-sample-10BT	boolq	0.4453	0.0087
finetuned	pythia-1b	prefix-tuning	fineweb-edu-sample-10BT	gsm8k	0.0212	0.0040
finetuned	pythia-1b	prefix-tuning	fineweb-edu-sample-10BT	piqa	0.6768	0.0109
finetuned	pythia-1b	prefix-tuning	starcoderdata-cpp	anli	0.3296	0.0144
finetuned	pythia-1b	prefix-tuning	starcoderdata-cpp	arc_easy	0.5572	0.0102
finetuned	pythia-1b	prefix-tuning	starcoderdata-cpp	bbh_boolean	0.1080	0.0197
finetuned	pythia-1b	prefix-tuning	starcoderdata-cpp	bbh_logical	0.1800	0.0243
finetuned	pythia-1b	prefix-tuning	starcoderdata-cpp	boolq	0.4896	0.0087
finetuned	pythia-1b	prefix-tuning	starcoderdata-cpp	gsm8k	0.0250	0.0043
finetuned	pythia-1b	prefix-tuning	starcoderdata-cpp	piqa	0.6785	0.0109
finetuned	pythia-1b	prefix-tuning	starcoderdata-java	anli	0.3245	0.0144
finetuned	pythia-1b	prefix-tuning	starcoderdata-java	arc_easy	0.5387	0.0102
finetuned	pythia-1b	prefix-tuning	starcoderdata-java	bbh_boolean	0.1920	0.0250
finetuned	pythia-1b	prefix-tuning	starcoderdata-java	bbh_logical	0.1640	0.0235
finetuned	pythia-1b	prefix-tuning	starcoderdata-java	boolq	0.5648	0.0087
finetuned	pythia-1b	prefix-tuning	starcoderdata-java	gsm8k	0.0190	0.0038
finetuned	pythia-1b	prefix-tuning	starcoderdata-java	piqa	0.6763	0.0109
finetuned	pythia-1b	prefix-tuning	starcoderdata-multi	anli	0.3177	0.0143
finetuned	pythia-1b	prefix-tuning	starcoderdata-multi	arc_easy	0.5551	0.0102
finetuned	pythia-1b	prefix-tuning	starcoderdata-multi	bbh_boolean	0.5000	0.0317
finetuned	pythia-1b	prefix-tuning	starcoderdata-multi	bbh_logical	0.2920	0.0288
finetuned	pythia-1b	prefix-tuning	starcoderdata-multi	boolq	0.5453	0.0087
finetuned	pythia-1b	prefix-tuning	starcoderdata-multi	gsm8k	0.0190	0.0038
finetuned	pythia-1b	prefix-tuning	starcoderdata-multi	piqa	0.6746	0.0109
finetuned	pythia-1b	prefix-tuning	starcoderdata-python	anli	0.3174	0.0143
finetuned	pythia-1b	prefix-tuning	starcoderdata-python	arc_easy	0.5480	0.0102
finetuned	pythia-1b	prefix-tuning	starcoderdata-python	bbh_boolean	0.0080	0.0056
finetuned	pythia-1b	prefix-tuning	starcoderdata-python	bbh_logical	0.3360	0.0299
finetuned	pythia-1b	prefix-tuning	starcoderdata-python	boolq	0.5153	0.0087
finetuned	pythia-1b	prefix-tuning	starcoderdata-python	gsm8k	0.0182	0.0037
finetuned	pythia-1b	prefix-tuning	starcoderdata-python	piqa	0.6708	0.0110

Continued on next page

Continued from previous page

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
finetuned	pythia-1b	vera	fineweb-edu-sample-10BT	anli	0.3241	0.0144
finetuned	pythia-1b	vera	fineweb-edu-sample-10BT	arc_easy	0.5909	0.0101
finetuned	pythia-1b	vera	fineweb-edu-sample-10BT	bbh_boolean	0.4760	0.0316
finetuned	pythia-1b	vera	fineweb-edu-sample-10BT	bbh_logical	0.3120	0.0294
finetuned	pythia-1b	vera	fineweb-edu-sample-10BT	boolq	0.5893	0.0086
finetuned	pythia-1b	vera	fineweb-edu-sample-10BT	gsm8k	0.0197	0.0038
finetuned	pythia-1b	vera	fineweb-edu-sample-10BT	piqa	0.7116	0.0106
finetuned	pythia-1b	vera	starcoderdata-cpp	anli	0.3241	0.0144
finetuned	pythia-1b	vera	starcoderdata-cpp	arc_easy	0.5884	0.0101
finetuned	pythia-1b	vera	starcoderdata-cpp	bbh_boolean	0.4520	0.0315
finetuned	pythia-1b	vera	starcoderdata-cpp	bbh_logical	0.3280	0.0298
finetuned	pythia-1b	vera	starcoderdata-cpp	boolq	0.5881	0.0086
finetuned	pythia-1b	vera	starcoderdata-cpp	gsm8k	0.0205	0.0039
finetuned	pythia-1b	vera	starcoderdata-cpp	piqa	0.7111	0.0106
finetuned	pythia-1b	vera	starcoderdata-java	anli	0.3227	0.0144
finetuned	pythia-1b	vera	starcoderdata-java	arc_easy	0.5884	0.0101
finetuned	pythia-1b	vera	starcoderdata-java	bbh_boolean	0.4480	0.0315
finetuned	pythia-1b	vera	starcoderdata-java	bbh_logical	0.3240	0.0297
finetuned	pythia-1b	vera	starcoderdata-java	boolq	0.5878	0.0086
finetuned	pythia-1b	vera	starcoderdata-java	gsm8k	0.0212	0.0040
finetuned	pythia-1b	vera	starcoderdata-java	piqa	0.7127	0.0106
finetuned	pythia-1b	vera	starcoderdata-multi	anli	0.3241	0.0144
finetuned	pythia-1b	vera	starcoderdata-multi	arc_easy	0.5905	0.0101
finetuned	pythia-1b	vera	starcoderdata-multi	bbh_boolean	0.4640	0.0316
finetuned	pythia-1b	vera	starcoderdata-multi	bbh_logical	0.3120	0.0294
finetuned	pythia-1b	vera	starcoderdata-multi	boolq	0.5859	0.0086
finetuned	pythia-1b	vera	starcoderdata-multi	gsm8k	0.0220	0.0040
finetuned	pythia-1b	vera	starcoderdata-multi	piqa	0.7106	0.0106
finetuned	pythia-1b	vera	starcoderdata-python	anli	0.3257	0.0144
finetuned	pythia-1b	vera	starcoderdata-python	arc_easy	0.5875	0.0101
finetuned	pythia-1b	vera	starcoderdata-python	bbh_boolean	0.4800	0.0317
finetuned	pythia-1b	vera	starcoderdata-python	bbh_logical	0.3200	0.0296
finetuned	pythia-1b	vera	starcoderdata-python	boolq	0.5878	0.0086
finetuned	pythia-1b	vera	starcoderdata-python	gsm8k	0.0220	0.0040
finetuned	pythia-1b	vera	starcoderdata-python	piqa	0.7111	0.0106
finetuned	pythia-2.8b	ia3	fineweb-edu-sample-10BT	anli	0.3395	0.0145
finetuned	pythia-2.8b	ia3	fineweb-edu-sample-10BT	arc_easy	0.6700	0.0096
finetuned	pythia-2.8b	ia3	fineweb-edu-sample-10BT	bbh_boolean	0.4280	0.0314
finetuned	pythia-2.8b	ia3	fineweb-edu-sample-10BT	bbh_logical	0.3760	0.0307
finetuned	pythia-2.8b	ia3	fineweb-edu-sample-10BT	boolq	0.6713	0.0082
finetuned	pythia-2.8b	ia3	fineweb-edu-sample-10BT	gsm8k	0.0250	0.0043
finetuned	pythia-2.8b	ia3	fineweb-edu-sample-10BT	piqa	0.7443	0.0102
finetuned	pythia-2.8b	ia3	starcoderdata-cpp	anli	0.3397	0.0146
finetuned	pythia-2.8b	ia3	starcoderdata-cpp	arc_easy	0.6726	0.0096
finetuned	pythia-2.8b	ia3	starcoderdata-cpp	bbh_boolean	0.4480	0.0315
finetuned	pythia-2.8b	ia3	starcoderdata-cpp	bbh_logical	0.3800	0.0308
finetuned	pythia-2.8b	ia3	starcoderdata-cpp	boolq	0.6697	0.0082
finetuned	pythia-2.8b	ia3	starcoderdata-cpp	gsm8k	0.0250	0.0043
finetuned	pythia-2.8b	ia3	starcoderdata-cpp	piqa	0.7448	0.0102
finetuned	pythia-2.8b	ia3	starcoderdata-java	anli	0.3433	0.0146
finetuned	pythia-2.8b	ia3	starcoderdata-java	arc_easy	0.6726	0.0096
finetuned	pythia-2.8b	ia3	starcoderdata-java	bbh_boolean	0.4560	0.0316
finetuned	pythia-2.8b	ia3	starcoderdata-java	bbh_logical	0.3640	0.0305
finetuned	pythia-2.8b	ia3	starcoderdata-java	boolq	0.6713	0.0082
finetuned	pythia-2.8b	ia3	starcoderdata-java	gsm8k	0.0250	0.0043
finetuned	pythia-2.8b	ia3	starcoderdata-java	piqa	0.7448	0.0102

Continued on next page

Continued from previous page

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
finetuned	pythia-2.8b	ia3	starcoderdata-multi	anli	0.3407	0.0146
finetuned	pythia-2.8b	ia3	starcoderdata-multi	arc_easy	0.6738	0.0096
finetuned	pythia-2.8b	ia3	starcoderdata-multi	bbh_boolean	0.4520	0.0315
finetuned	pythia-2.8b	ia3	starcoderdata-multi	bbh_logical	0.3600	0.0304
finetuned	pythia-2.8b	ia3	starcoderdata-multi	boolq	0.6709	0.0082
finetuned	pythia-2.8b	ia3	starcoderdata-multi	gsm8k	0.0227	0.0041
finetuned	pythia-2.8b	ia3	starcoderdata-multi	piqa	0.7448	0.0102
finetuned	pythia-2.8b	lora	fineweb-edu-sample-10BT	anli	0.3304	0.0145
finetuned	pythia-2.8b	lora	fineweb-edu-sample-10BT	arc_easy	0.6334	0.0099
finetuned	pythia-2.8b	lora	fineweb-edu-sample-10BT	bbh_boolean	0.5000	0.0317
finetuned	pythia-2.8b	lora	fineweb-edu-sample-10BT	bbh_logical	0.3600	0.0304
finetuned	pythia-2.8b	lora	fineweb-edu-sample-10BT	boolq	0.6394	0.0084
finetuned	pythia-2.8b	lora	fineweb-edu-sample-10BT	gsm8k	0.0167	0.0035
finetuned	pythia-2.8b	lora	fineweb-edu-sample-10BT	piqa	0.7176	0.0105
finetuned	pythia-2.8b	lora	starcoderdata-cpp	anli	0.3366	0.0145
finetuned	pythia-2.8b	lora	starcoderdata-cpp	arc_easy	0.6279	0.0099
finetuned	pythia-2.8b	lora	starcoderdata-cpp	bbh_boolean	0.5520	0.0315
finetuned	pythia-2.8b	lora	starcoderdata-cpp	bbh_logical	0.3920	0.0309
finetuned	pythia-2.8b	lora	starcoderdata-cpp	boolq	0.6428	0.0084
finetuned	pythia-2.8b	lora	starcoderdata-cpp	gsm8k	0.0167	0.0035
finetuned	pythia-2.8b	lora	starcoderdata-cpp	piqa	0.7263	0.0104
finetuned	pythia-2.8b	lora	starcoderdata-java	anli	0.3397	0.0145
finetuned	pythia-2.8b	lora	starcoderdata-java	arc_easy	0.6460	0.0098
finetuned	pythia-2.8b	lora	starcoderdata-java	bbh_boolean	0.4960	0.0317
finetuned	pythia-2.8b	lora	starcoderdata-java	bbh_logical	0.3440	0.0301
finetuned	pythia-2.8b	lora	starcoderdata-java	boolq	0.6165	0.0085
finetuned	pythia-2.8b	lora	starcoderdata-java	gsm8k	0.0167	0.0035
finetuned	pythia-2.8b	lora	starcoderdata-java	piqa	0.7193	0.0105
finetuned	pythia-2.8b	lora	starcoderdata-multi	anli	0.3398	0.0145
finetuned	pythia-2.8b	lora	starcoderdata-multi	arc_easy	0.6343	0.0099
finetuned	pythia-2.8b	lora	starcoderdata-multi	bbh_boolean	0.4200	0.0313
finetuned	pythia-2.8b	lora	starcoderdata-multi	bbh_logical	0.3760	0.0307
finetuned	pythia-2.8b	lora	starcoderdata-multi	boolq	0.6187	0.0085
finetuned	pythia-2.8b	lora	starcoderdata-multi	gsm8k	0.0197	0.0038
finetuned	pythia-2.8b	lora	starcoderdata-multi	piqa	0.7263	0.0104
finetuned	pythia-2.8b	lora	starcoderdata-python	anli	0.3384	0.0145
finetuned	pythia-2.8b	lora	starcoderdata-python	arc_easy	0.6494	0.0098
finetuned	pythia-2.8b	lora	starcoderdata-python	bbh_boolean	0.5040	0.0317
finetuned	pythia-2.8b	lora	starcoderdata-python	bbh_logical	0.3800	0.0308
finetuned	pythia-2.8b	lora	starcoderdata-python	boolq	0.6018	0.0086
finetuned	pythia-2.8b	lora	starcoderdata-python	gsm8k	0.0227	0.0041
finetuned	pythia-2.8b	lora	starcoderdata-python	piqa	0.7296	0.0104
finetuned	pythia-2.8b	prefix-tuning	fineweb-edu-sample-10BT	anli	0.3315	0.0145
finetuned	pythia-2.8b	prefix-tuning	fineweb-edu-sample-10BT	arc_easy	0.3590	0.0098
finetuned	pythia-2.8b	prefix-tuning	fineweb-edu-sample-10BT	bbh_boolean	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	fineweb-edu-sample-10BT	bbh_logical	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	fineweb-edu-sample-10BT	boolq	0.4468	0.0087
finetuned	pythia-2.8b	prefix-tuning	fineweb-edu-sample-10BT	gsm8k	0.0190	0.0038
finetuned	pythia-2.8b	prefix-tuning	fineweb-edu-sample-10BT	piqa	0.5903	0.0115
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-cpp	anli	0.3323	0.0145
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-cpp	arc_easy	0.4049	0.0101
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-cpp	bbh_boolean	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-cpp	bbh_logical	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-cpp	boolq	0.5437	0.0087
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-cpp	gsm8k	0.0061	0.0021
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-cpp	piqa	0.6197	0.0113

Continued on next page

Continued from previous page

Type	Model	PEFT	Dataset	Task	Accuracy	Stderr
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-multi	anli	0.3315	0.0145
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-multi	arc_easy	0.3813	0.0100
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-multi	bbh_boolean	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-multi	bbh_logical	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-multi	boolq	0.5147	0.0087
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-multi	gsm8k	0.0114	0.0029
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-multi	piqa	0.5898	0.0115
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-python	anli	0.3283	0.0144
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-python	arc_easy	0.4095	0.0101
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-python	bbh_boolean	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-python	bbh_logical	0.0000	0.0000
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-python	boolq	0.4844	0.0087
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-python	gsm8k	0.0083	0.0025
finetuned	pythia-2.8b	prefix-tuning	starcoderdata-python	piqa	0.5974	0.0114
finetuned	pythia-2.8b	vera	fineweb-edu-sample-10BT	anli	0.3402	0.0146
finetuned	pythia-2.8b	vera	fineweb-edu-sample-10BT	arc_easy	0.6730	0.0096
finetuned	pythia-2.8b	vera	fineweb-edu-sample-10BT	bbh_boolean	0.4600	0.0316
finetuned	pythia-2.8b	vera	fineweb-edu-sample-10BT	bbh_logical	0.3680	0.0306
finetuned	pythia-2.8b	vera	fineweb-edu-sample-10BT	boolq	0.6722	0.0082
finetuned	pythia-2.8b	vera	fineweb-edu-sample-10BT	gsm8k	0.0250	0.0043
finetuned	pythia-2.8b	vera	fineweb-edu-sample-10BT	piqa	0.7470	0.0101
finetuned	pythia-2.8b	vera	starcoderdata-cpp	anli	0.3420	0.0146
finetuned	pythia-2.8b	vera	starcoderdata-cpp	arc_easy	0.6751	0.0096
finetuned	pythia-2.8b	vera	starcoderdata-cpp	bbh_boolean	0.4280	0.0314
finetuned	pythia-2.8b	vera	starcoderdata-cpp	bbh_logical	0.3560	0.0303
finetuned	pythia-2.8b	vera	starcoderdata-cpp	boolq	0.6709	0.0082
finetuned	pythia-2.8b	vera	starcoderdata-cpp	gsm8k	0.0265	0.0044
finetuned	pythia-2.8b	vera	starcoderdata-cpp	piqa	0.7470	0.0101
finetuned	pythia-2.8b	vera	starcoderdata-java	anli	0.3427	0.0146
finetuned	pythia-2.8b	vera	starcoderdata-java	arc_easy	0.6742	0.0096
finetuned	pythia-2.8b	vera	starcoderdata-java	bbh_boolean	0.4360	0.0314
finetuned	pythia-2.8b	vera	starcoderdata-java	bbh_logical	0.3720	0.0306
finetuned	pythia-2.8b	vera	starcoderdata-java	boolq	0.6725	0.0082
finetuned	pythia-2.8b	vera	starcoderdata-java	gsm8k	0.0243	0.0042
finetuned	pythia-2.8b	vera	starcoderdata-java	piqa	0.7470	0.0101
finetuned	pythia-2.8b	vera	starcoderdata-multi	anli	0.3421	0.0146
finetuned	pythia-2.8b	vera	starcoderdata-multi	arc_easy	0.6734	0.0096
finetuned	pythia-2.8b	vera	starcoderdata-multi	bbh_boolean	0.4240	0.0313
finetuned	pythia-2.8b	vera	starcoderdata-multi	bbh_logical	0.3720	0.0306
finetuned	pythia-2.8b	vera	starcoderdata-multi	boolq	0.6722	0.0082
finetuned	pythia-2.8b	vera	starcoderdata-multi	gsm8k	0.0250	0.0043
finetuned	pythia-2.8b	vera	starcoderdata-multi	piqa	0.7476	0.0101

A.2 Configuration-Level Task Breakdown

Figure A.1: Faceted bar plot of task-level configuration breakdown of Δ^* for RQ5. Each panel shows Δ^* across all model–PEFT configurations; rows are grouped by base model and PEFT method, and within each group configurations are ordered by code dataset. Positive values indicate code advantage over the matched natural-language baseline; negative values indicate text advantage. All panels share the same x-axis limits and use a symmetric log scale to show both small and large effects clearly.

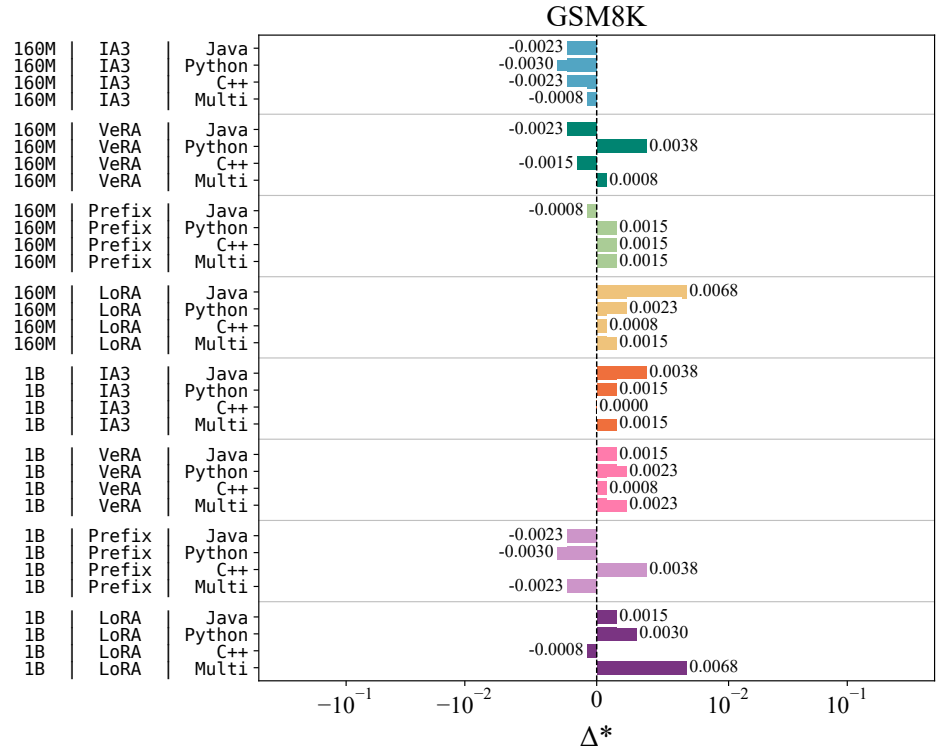

Figure A.1: Task-level configuration breakdown of Δ^* for RQ5 (continued).

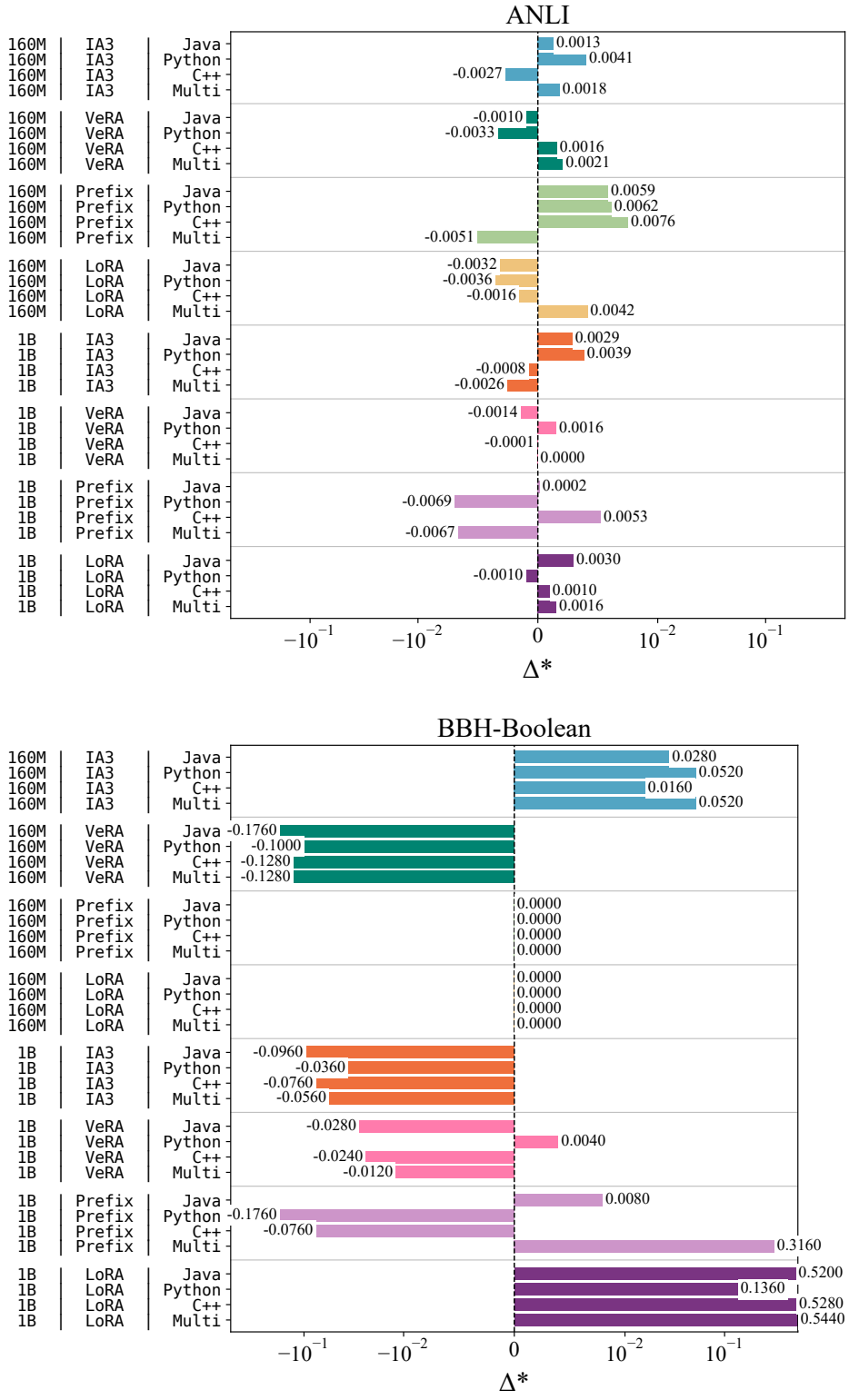

Figure A.1: Task-level configuration breakdown of Δ^* for RQ5 (continued).

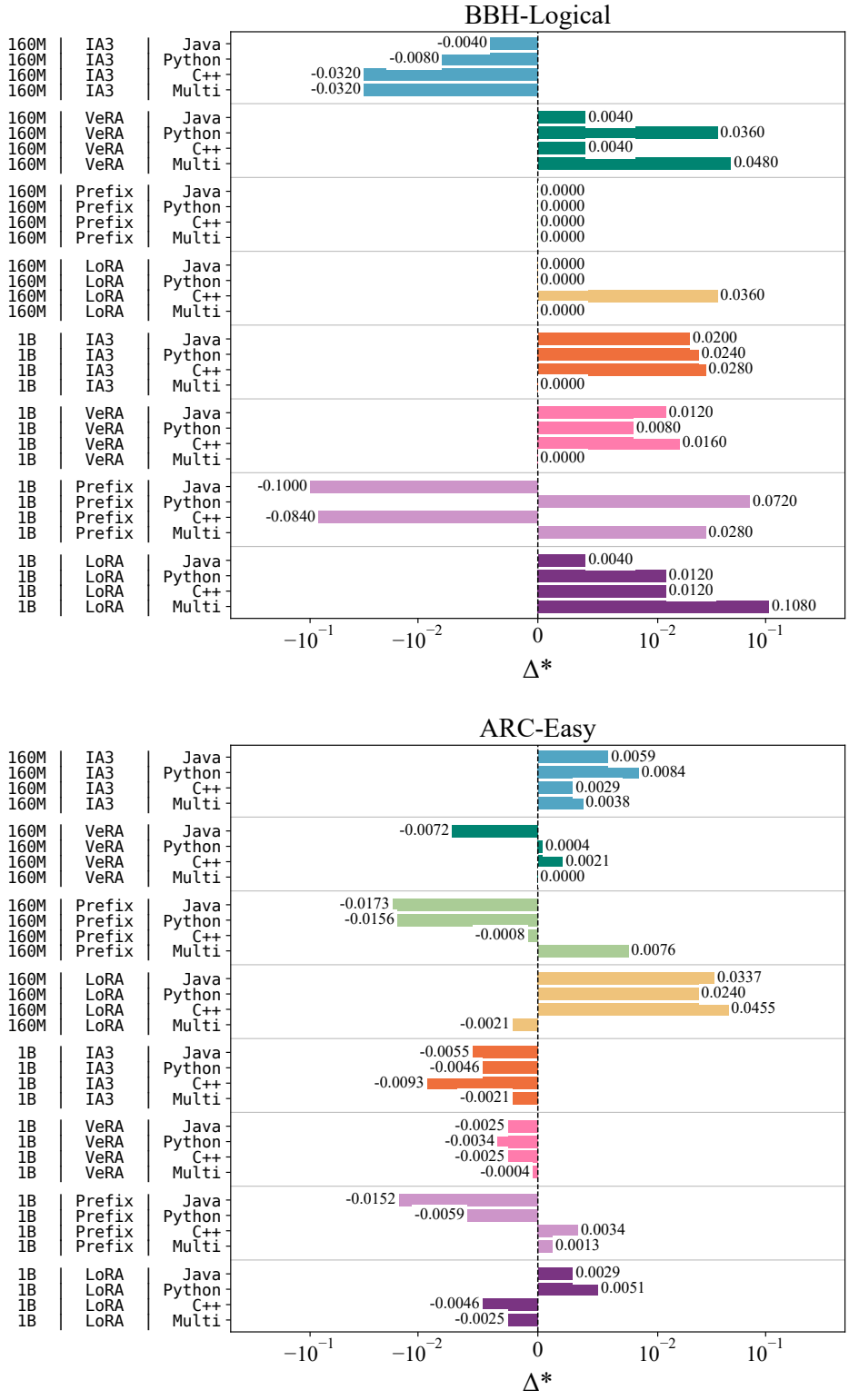

Figure A.1: Task-level configuration breakdown of Δ^* for RQ5 (continued).